HRMS Module

Study, Page Connections & Flowcharts

ProgBiz ERP — https://hrms-erp.progbiz.in · tenant "Hrms"

80 pages crawled live · 6 sub-modules · prepared for Playwright automation

20 July 2026

Contents

HRMS Module — Overview, Structure & Flowcharts

> ***App:*** *https://hrms-erp.progbiz.in* · ***Tenant:*** *company code `Hrms` · test user `vismaya`*
>
> ***Prepared for:*** *Playwright automation code preparation (module study phase)*
>
> ***Source of truth:*** *live crawl of all 80 HRMS pages on 2026-07-20 — raw captures in `hrms/data/pages/`, screenshots in `hrms/screenshots/`*

1. What the HRMS module is

The HRMS module of the ProgBiz ERP covers the **entire employee lifecycle**: hiring (Recruitment), joining (Onboarding), the employee master and payroll inputs (Core HR), day-to-day time tracking (Attendance), absence handling (Leave Management), and an employee-facing portal (My Workspace / ESS). A generic **approval-workflow engine** (`/approvals` + `/approval/config`) threads through all of it — leave requests, salary revisions, regularizations, profile-change requests and encashments all land in the same approval inbox.

The login form uses the same 3-field pattern as the other ERP builds: `#companycode`, `#signin-username`, `#signin-password`. After login the user lands on `/home`.

2. Sub-module map (menu tree)

```
HRMS
├── Core HR
│   ├── Employee ....................... /employees
│   ├── Section ........................ /sections
│   ├── Worker Directory ............... /worker-directory        (Cards | Org Chart)
│   ├── Salary Revisions ............... /salary-revisions
│   ├── Employee Salary Process ........ /employee-salary-process
│   ├── Employee Deductions ............ /employee-deduction
│   ├── Employee Remarks ............... /employee-remark
│   ├── Probation Dashboard ............ /hrms/probation
│   ├── Probation Templates ............ /hrms/probation-templates
│   ├── Probation Report ............... /hrms/probation-report
│   ├── Resigned Employees ............. /resigned-employees
│   ├── Employee Excel Import .......... /upload-employee
│   ├── Letters
│   │   ├── Letter Templates ........... /letters/templates       (+ /letters/fields)
│   │   └── Generate Letter ............ /letters/generate
│   ├── Approvals
│   │   ├── My Approvals ............... /approvals               (workflow inbox)
│   │   └── Approval Config ............ /approval/config         (chain builder)
│   └── Reports
│       ├── Deduction Report ........... /employee-deduction-report
│       └── Remark Report .............. /employee-remark-report
├── Recruitment
│   ├── Job Requisitions ............... /requisition-list
│   ├── Job Board ...................... /vacancy-list            (Job Openings | Candidates | Talent Pools)
│   ├── Current Openings ............... /current-openings        (public "Join Our Team" careers page)
│   ├── Job Applications ............... /job-applications-list
│   ├── Candidates ..................... /candidates              (New | In Progress | Shortlisted | Selected |
Rejected)
│   ├── Assessments .................... /assessment-list
│   ├── Interview Schedules ............ /interview-schedules
│   ├── Offers ......................... /offer-list
│   ├── Pipeline ▸ Pipeline Board ...... /recruitment-pipeline    (Kanban, Configure Stages, Score)
│   ├── Communication Templates ........ /communication-templates
│   ├── Talent Pool .................... /talent-pool
│   ├── Settings
│   │   ├── Candidate Status ........... /candidate-status
│   │   └── Interview Rounds ........... /interview-rounds
```

```
│   └── Onboarding
│       ├── Onboarding Templates ......... /onboarding-templates
│       └── Onboarding Pipeline .......... /onboarding-pipeline
├── Attendance
│   ├── Shifts & Rules ................... /shifts
│   ├── Shift Roster ..................... /shift-roster
│   ├── Attendance Log ................... /attendance-log
│   ├── Data from Device ................. /data-from-device        (biometric punches)
│   ├── Add Visit Report ................. /add-visit-report        (field-visit punches)
│   ├── Regularization ................... /regularization
│   ├── Overtime Approval ................ /overtime-approval
│   ├── Attendance Finalization .......... /attendance-finalization (pay-cycle runs)
│   ├── Geofences ........................ /geofences
│   ├── Timesheet ........................ /timesheet               (attendance hrs vs task hrs)
│   ├── Attendance Report Pack ........... /attendance-report-pack
│   └── Approvals
│       ├── Approval Operation ........... /approval-operation      (worked/late/OT hour approval)
│       ├── Approval Operation Report .... /approval-operation-report
│       ├── Approval Absent .............. /approval-absent
│       └── Approval Absent Report ....... /approval-absent-report
├── Leave Management
│   ├── Leave Types ...................... /leave-types
│   ├── Leave Patterns ................... /leave-patterns
│   ├── Leave Policy ..................... /leave-policy
│   ├── Leave Assignment ................. /leave-assignment-list
│   ├── Leave Request .................... /leave-request-list
│   ├── Leave Approval ................... /leave-approval
│   ├── My Leave Policy .................. /my-leave-policy
│   ├── Leave Balances ................... /leave-balances          (Run Accrual)
│   ├── Leave Ledger ..................... /leave-ledger
│   ├── Attendance Sync .................. /leave-attendance-sync   (LOP recalculation)
│   ├── Leave Encashment ................. /leave-encashment
│   ├── Encashment Approval .............. /leave-encashment-approval
│   ├── Leave Delegation ................. /leave-delegation
│   ├── Employee Handover ................ /employee-handover
│   ├── Comp-Offs ........................ /comp-offs
│   ├── Comp-Off Management .............. /comp-off-management
│   ├── Holidays ......................... /holiday-list            (Calendar | Export)
│   ├── Holiday Assignment ............... /holiday-assignment-list
│   └── Leave Analytics
│       ├── Leave Reports ................ /leave-reports           (Register | Balance | Utilization)
│       ├── Absence Analytics ............ /absence-analytics       (Bradford Factor)
│       └── Leave Calendar ............... /leave-calendar
└── My Workspace (Employee Self-Service)
    ├── My Workspace ..................... /ess                     (self-service dashboard)
    ├── My Profile ....................... /ess/profile
    ├── My Requests ...................... /ess/requests            (profile change requests)
    ├── My Leave ......................... /ess/leave               (balances + apply)
    ├── My Handover ...................... /my-handover
    ├── My Attendance .................... /ess/attendance          (Regularize | Raise OT)
    ├── My Locations ..................... /ess/locations           (geo work locations, map)
    ├── My Documents ..................... /ess/documents
    ├── My Letters ....................... /ess/letters
    ├── My Pay ........................... /ess/payslips
    └── My Probation ..................... /ess/probation
```

Related org masters (under the separate **Master** menu, referenced by HRMS forms): `department`, `designations`, `teams`, `business-contacts`.

3. HRMS module map (flowchart)

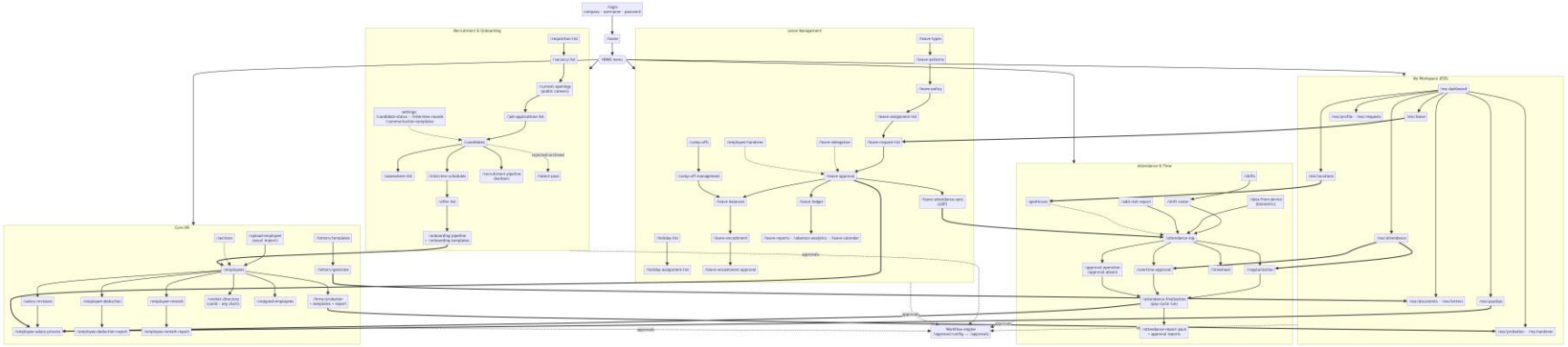

Figure 1: 3. HRMS module map (flowchart)

4. End-to-end process flows

4.1 Hire-to-Employee (Recruitment → Onboarding → Core HR)

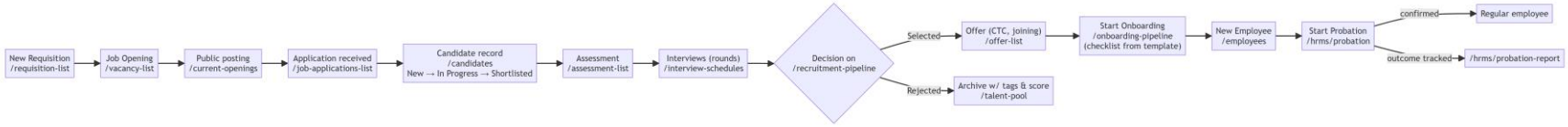

Figure 2: 4.1 Hire-to-Employee (Recruitment → Onboarding → Core HR)

Supporting config: `/candidate-status` (follow-up stages), `/interview-rounds` (round order), `/communication-templates` (candidate emails), `/onboarding-templates` (joining checklist).

4.2 Daily attendance cycle

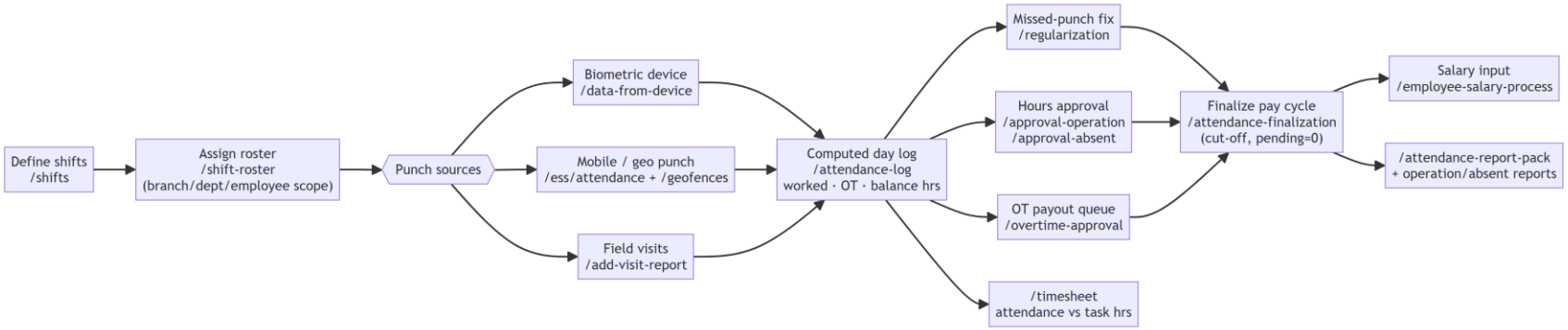

Figure 3: 4.2 Daily attendance cycle

4.3 Leave lifecycle

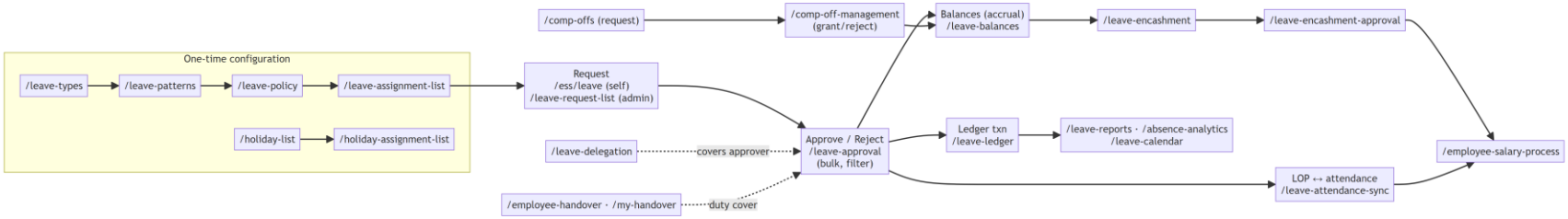

Figure 4: 4.3 Leave lifecycle

4.4 Salary & document admin (Core HR)

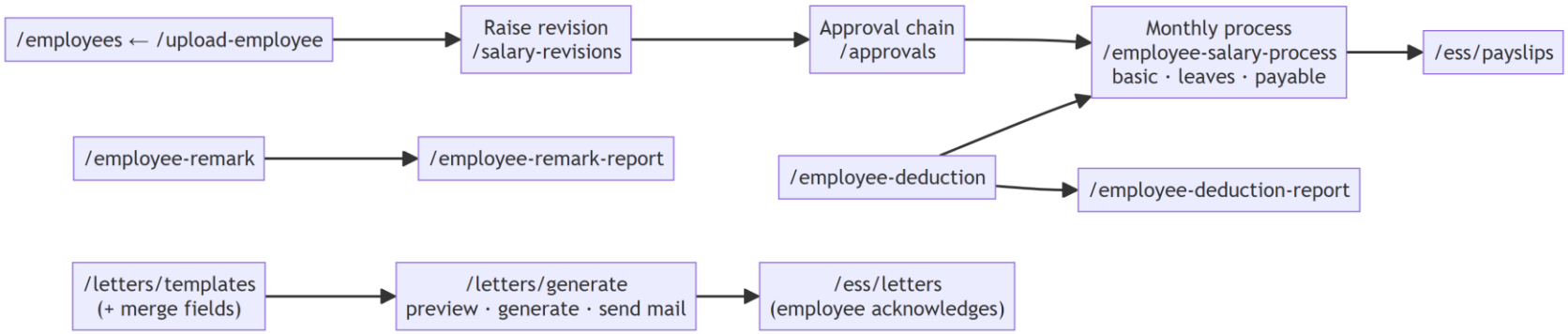

Figure 5: 4.4 Salary & document admin (Core HR)

4.5 Approval workflow engine (cross-cutting)

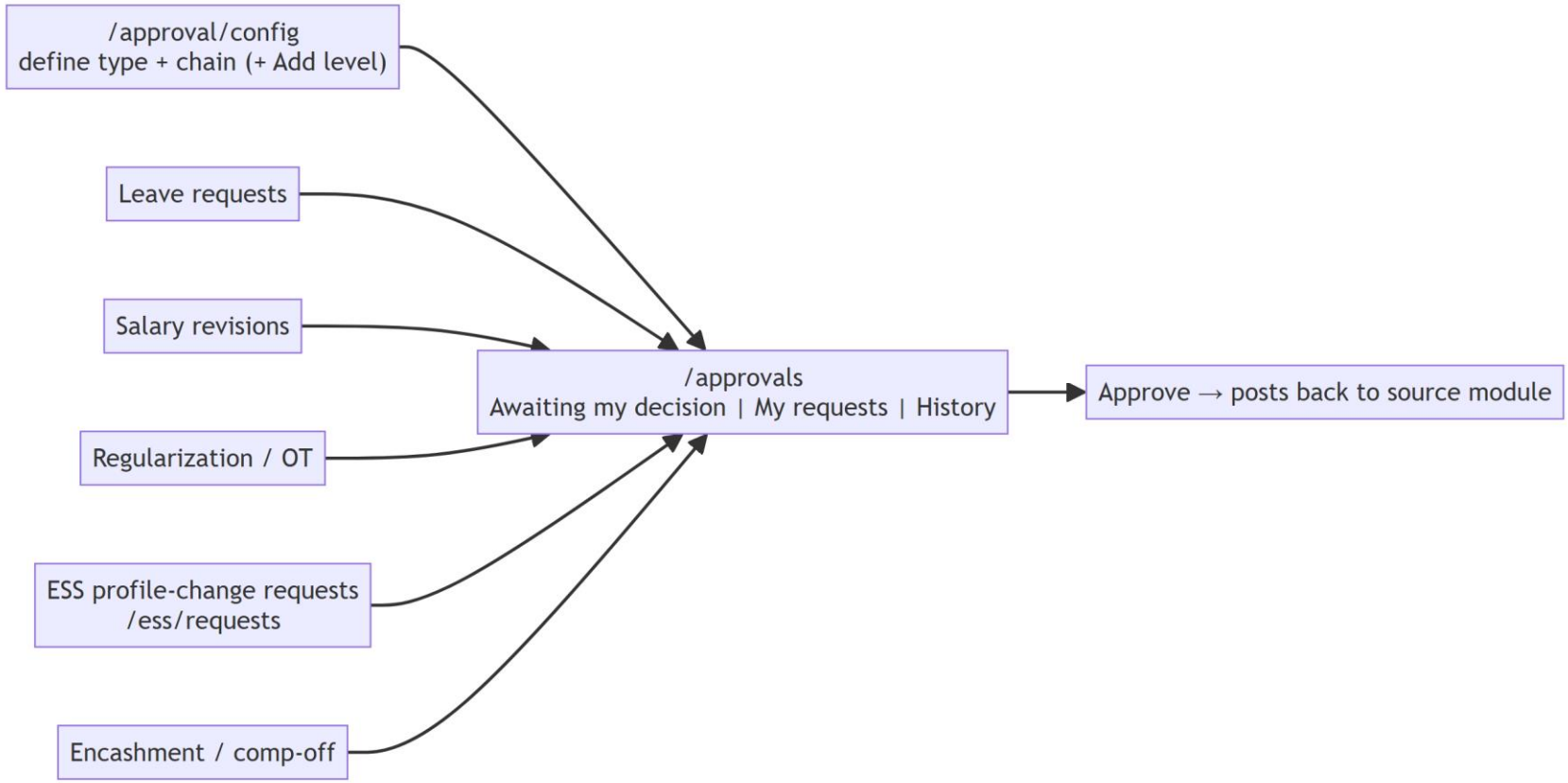

Figure 6: 4.5 Approval workflow engine (cross-cutting)

5. ESS ↔ admin page pairing (page connections)

Employee (ESS) page	Admin counterpart	Connection
`/ess/leave`	`/leave-request-list`, `/leave-approval`	request → approval → ledger/balance
`/ess/attendance` (`Regularize`, `Raise OT`)	`/regularization`, `/overtime-approval`	self-raised corrections land in admin queues
`/ess/locations` (`Submit for approval`)	`/geofences`	approved location becomes a geofence punch-zone
`/ess/documents`	employee record `/employees`	doc upload w/ expiry tracked
`/ess/letters`	`/letters/generate`	HR generates → employee acknowledges
`/ess/payslips`	`/employee-salary-process`	processed pay → payslip rows
`/ess/probation`	`/hrms/probation`	employee view of own probation reviews
`/my-handover`	`/employee-handover`	self vs admin handover setup
`/ess/requests`	`/approvals`	profile change requests through workflow
`/ess/profile`	`/employees` (record)	master data view

6. Automation-relevant observations (from the crawl)

1. **Login:** same selectors as other tenants — `#companycode`, `#signin-username`, `#signin-password`, `button[type=submit]`; success = URL leaves `/login`, lands on `/home`.
2. **Two page archetypes dominate:** (a) *config/list pages* with a `New X` button opening a modal/inline form + a data grid; (b) *inline-form + grid* pages (form card on top, e.g. Sections, Leave Types, Regularization, Handover) with `Save`/`Clear` buttons.
3. **Filter-first report pages:** most report pages (`attendance-log`, `approval-*`, `leave-ledger`, `*-report`) render an **empty grid until `Filter`/`View Report`/`Run Report` is clicked** — tests must apply filters before asserting rows.
4. **Tabbed status pages:** `/candidates` (5 status tabs with counts), `/vacancy-list` (3 tabs), `/approvals` (3 tabs) — tab state is a key assertion target.
5. **Non-grid pages needing special handling:** `/recruitment-pipeline` and `/onboarding-pipeline` (kanban boards), `/leave-calendar` + `/holiday-list` (calendar views), `/worker-directory` (cards/org-chart), `/ess/locations` (Leaflet map), `/current-openings` (public page, likely reachable without auth — verify).
6. **Known label bugs to not "fix" in assertions:** `/employee-remark` page header reads **"Employee Deduction"** (copy-paste bug); `/add-visit-report` header is misspelled **"Add Vist Report"**; `/upload-employee` shows a stray "Leave Request" card title.
7. **Excel touchpoints:** `/upload-employee` (sample file `EmployeeExcelImport.xlsx`, `Excel Rules` dialog), exports on holiday list, probation report, leave ledger, attendance report pack.

8. **Data seeding order for E2E tests** mirrors the config chains: shifts → roster before any attendance assertions; leave types → patterns → policy → assignment before any leave request; requisition → opening before candidates/offers.

7. Folder layout

Path	Contents
`hrms/docs/`	This overview + per-sub-module deep dives (`01_CORE_HR.md` … `05_ESS_MY_WORKSPACE.md`)
`hrms/data/nav.json`	Full navigation capture (all anchors, sidebar tree, app shell)
`hrms/data/pages/`	One JSON per page: headers, tabs, buttons, table columns, inputs, links
`hrms/data/summary.json` / `summary.txt`	Merged per-group summaries
`hrms/screenshots/`	Login, landing, expanded menu + every page per group
`hrms/exploration/`	Re-runnable crawl scripts (`01_login_and_nav.js`, `02_crawl_pages.js <group>`, `03_summarize.js`)

Deep Dive — Core HR

Overview

The Core HR sub-module of the HRMS ERP (https://hrms-erp.progbiz.in) is the system of record for the employee lifecycle. It covers employee master data (the `/employees` register, the `/sections` department-section master, and bulk onboarding via `/upload-employee`), a browsable `/worker-directory` with card and org-chart views, and a separation register at `/resigned-employees`. Compensation is handled through `/salary-revisions` (raise a gross-salary revision that carries a Status, implying an approval step), `/employee-salary-process` (monthly branch-wise payable computation), and `/employee-deduction` (ad-hoc deduction entries), with matching report pages for deductions and remarks. A probation suite (`/hrms/probation`, `/hrms/probation-templates`, `/hrms/probation-report`) tracks employees on probation from template definition through checkpoint reviews to a confirmed/terminated outcome. HR correspondence is produced through `/letters/templates` and `/letters/generate` using merge fields. Cross-cutting all of this is the workflow engine: `/approval/config` defines multi-level approval chains per document type (including `Salary Revision` and `Probation Decision`), and `/approvals` is the inbox where approvers action pending items. On the crawled tenant most grids were empty, so every list page's empty-state text is captured below — useful for zero-data assertions in Playwright.

Page Index

#	Page	Route	Purpose
1	Employees	`/employees`	Employee master register; create and manage employee records
2	Sections	`/sections`	Master data: define sections under departments
3	Worker Directory	`/worker-directory`	Browse employees as cards or an org chart
4	Salary Revisions	`/salary-revisions`	Raise gross-salary revisions and view revision history/status
5	Employee Salary Process	`/employee-salary-process`	Compute and save monthly payable amounts per branch
6	Employee Deduction	`/employee-deduction`	Record an ad-hoc salary deduction for a staff member
7	Employee Remark	`/employee-remark`	Record a dated remark/note against a staff member
8	Probation Dashboard	`/hrms/probation`	KPI dashboard and list of employees currently on probation
9	Probation Templates	`/hrms/probation-templates`	Define probation duration/checkpoint/criteria templates
10	Probation Report	`/hrms/probation-report`	Date-ranged probation outcome report with Excel export
11	Resigned Employees	`/resigned-employees`	Register of employees who have resigned
12	Employee Excel Import	`/upload-employee`	Bulk-create employees from an Excel file
13	Letter Templates	`/letters/templates`	Manage HR letter templates (with merge fields)

#	Page	Route	Purpose
14	Generate Letter	`/letters/generate`	Generate/preview/email a letter for an employee from a template
15	My Approvals	`/approvals`	Approver inbox: pending decisions, own requests, history
16	Approval Configuration	`/approval/config`	Define per-type multi-level approval workflows
17	Employee Deduction Reports	`/employee-deduction-report`	Filterable report over recorded deductions
18	Employee Remark Reports	`/employee-remark-report`	Filterable report over recorded remarks

Pages

Employees (`/employees`)

- **Purpose** — Employee master register. Lists all employees with their code, department and designation, and is the entry point for creating a new employee record.
- **UI elements** — Buttons: `Filter`, `New Employee`. Checkbox `Include archived` (`input#incl-archived`). Card: `Employees`.
- **Data grid** — Columns: `Sl.No`, `Employee Code`, `Employee Name`, `Department Name`, `Designation`, `Status`, `Actions`. One row = one employee record.
- **Connections** — Employee records created here populate `/worker-directory`, are selectable in the Employee dropdowns of `/salary-revisions`, `/employee-deduction`, `/employee-remark`, `/letters/generate`, and appear in `/employee-salary-process` payroll grids. Bulk alternative: `/upload-employee`. Separated employees end up in `/resigned-employees`; probation for new hires is tracked in `/hrms/probation`.
- **Automation notes** — `New Employee` likely opens a create form/modal; `Filter` likely toggles a filter panel. The `#incl-archived` checkbox is a stable selector for the archived toggle and implies soft-delete/archive behavior (test that archived employees appear only when it is checked). Grid rendered with headers but 0 rows on the crawl — assert on the header row for smoke tests.

Sections (`/sections`)

- **Purpose** — Master-data maintenance page for sections: create a section under a department and view/edit the existing department-section list.
- **UI elements** — Cards: `Sections` (list) and `New Section` (form). Form fields: `Department :` (native `select`, default option `Choose`), `Section Name*` (`input#sectionname`, required). Buttons: `Save`, `Clear`.
- **Data grid** — Columns: `SlNo`, `Department Name`, `Section Name`, `Action`. One row = one section mapped to a department.
- **Connections** — Departments/sections defined here back the `Department Name` column on `/employees` and the department dimension used in directory search on `/worker-directory`.
- **Automation notes** — Single-page master (form + grid side by side), no modal. `Section Name*` is mandatory — assert validation on empty save. `#sectionname` is a stable input selector; the department `select` has no id, so target it by its `Department :` label. `Clear` resets the form.

Worker Directory (`/worker-directory`)

- **Purpose** — Read-only employee directory for browsing people, with a card view and an organisation-chart view.
- **UI elements** — View toggle buttons (top-right): `Cards` | `Org Chart`. Filters: `Branch` select (default `-- All branches --`), search text box (placeholder `Name, designation or department`), `Search` button.
- **Data grid** — None; results render as employee cards (or an org chart in the `Org Chart` view).
- **Connections** — Displays records maintained in `/employees` / imported via `/upload-employee`. Org-chart hierarchy implies reporting-line data captured on the employee record.
- **Automation notes** — Empty state text: `No employees match the current filters.` — appears when no employees match (or on an empty tenant), good for a negative-search assertion. Search is button-triggered (`Search`), not live-typed. The `Cards`/`Org Chart` toggle switches rendering mode — verify both views. Search input is identified by its placeholder; the branch `select` has no id.

Salary Revisions (`/salary-revisions`)

- **Purpose** — Raise a salary (gross) revision for an employee and track all revisions with their approval status.
- **UI elements** — Card `Raise A Revision` with fields: `Branch` select (`-- Select branch --`), `Employee` select (`-- Select employee --`), `New Gross` (number input), `Effective Date` (date input), `Reason` (textarea). Button: `Raise Revision`. Card `Revision History` holds the grid.
- **Data grid** — Columns: `Employee`, `Branch`, `Effective`, `Old`, `New`, `%`, `Status`. One row = one salary revision (old vs new gross, % change, workflow status). Empty-state row: `No revisions yet.`
- **Connections** — Employee list comes from `/employees` (filtered by Branch). The `Status` column ties into the workflow engine: a `Salary Revision` approval type exists in `/approval/config`, and pending revisions surface for approvers in `/approvals`. Approved revisions change the gross used by `/employee-salary-process`.
- **Automation notes** — Branch select likely gates the Employee select (cascading dropdowns) — select branch first in tests. `New Gross` is `type=number`, `Effective Date` is `type=date`. The history table renders `No revisions yet.` as a single row (rowCount 1), not a separate empty-state element — assert on cell text. After `Raise Revision`, expect a new row with a `Status` (pending if a workflow is configured, auto-approved if not — see `/approval/config` "No levels = direct").

Employee Salary Process (`/employee-salary-process`)

- **Purpose** — Monthly payroll processing: pick branch/year/month, review each staff member's basic salary, leave count and computed payable amount, then save the run.
- **UI elements** — Filters: `Branch` select (`select#assignedToBranch`, default `All`), `Salary Year :` select and `Salary Month :` select (January–December; both selects share `name="narration-typ"`). Button: `Save`.
- **Data grid** — Columns: `Staff Name`, `Basic Salary`, `No of Leave`, `Payable Amount`. One row = one employee's salary computation for the chosen month. Empty-state row: `No Data`.
- **Connections** — Rows are employees from `/employees` for the selected branch; `Basic Salary` reflects the current (revised) salary from `/salary-revisions`; `No of Leave` is fed by the leave/attendance sub-module; deductions recorded in `/employee-deduction` logically reduce payable amounts.

- **Automation notes** — Grid shows `No Data` until branch/year/month yield results — treat year+month as effectively mandatory filters. `#assignedToBranch` is a stable selector; the year/month selects share a duplicate `name="narration-typ"`, so disambiguate by position or label, not by name. `Save` persists the whole displayed run (no per-row save button captured).

Employee Deduction (`/employee-deduction`)

- **Purpose** — Entry form to record a salary deduction against a staff member (type, amount, date, payment mode, details).
- **UI elements** — Card `Employee Deduction` with fields: `Branch` select (`select#branch`, default `Choose`), `Staff` select (`select#employee`, default `Choose`), `Date` (`input#enquiry-date`, `type=date`, placeholder `Enter Date`), `Deduction Type *` (typeahead text input `#building-type`, placeholder `Enter deduction type and search`), `Pay Using` select (default `Choose`; captured with duplicate `id="employee"`), `Amount :` (`input#amount`), `Details` (textarea `#details-text-area`). Buttons: `Save`, `Cancel`.
- **Data grid** — None (pure entry form).
- **Connections** — Staff options come from `/employees` (branch-filtered). Saved deductions are queried on `/employee-deduction-report` and logically reduce the `Payable Amount` in `/employee-salary-process`.
- **Automation notes** — `Deduction Type *` is mandatory and is a search-as-you-type field, not a select — type a value and pick from suggestions. Beware duplicate DOM ids: both the `Staff` select and the `Pay Using` select were captured as `#employee` — use label-relative or nth-of selectors. Non-semantic ids reused from other modules (`#enquiry-date`, `#building-type`) are stable but misleading — prefer label-based locators for readability.

Employee Remark (`/employee-remark`)

- **Purpose** — Entry form to record a dated remark (disciplinary/performance/general note) against a staff member.
- **UI elements** — Fields: `Branch` select (`select#branch`, default `Choose`), `Staff` select (`select#employee`, default `Choose`), `Date` (`input#enquiry-date`, `type=date`, placeholder `Enter Date`), `Details` (textarea `#details-text-area`). Buttons: `Save`, `Cancel`.
- **Data grid** — None (pure entry form).
- **Connections** — Staff options come from `/employees`. Saved remarks are reported on `/employee-remark-report`.
- **Automation notes** — Known label bug: the page header, breadcrumb and card title all read `Employee Deduction` even though the route is `/employee-remark` and the form is the remark form (no deduction-type/amount fields). Do not assert the page title equals "Employee Remark" on this build — assert on the URL plus the reduced field set instead. Same shared ids as the deduction form (`#branch`, `#employee`, `#enquiry-date`, `#details-text-area`).

Probation Dashboard (`/hrms/probation`)

- **Purpose** — Landing dashboard for probation management: KPI tiles plus the live list of employees currently on probation with review scheduling.
- **UI elements** — Header actions: `Report` (link to `/hrms/probation-report`), `Templates` (link to `/hrms/probation-templates`), `Start Probation` button. KPI tiles: `On Probation`, `Reviews Due (7d)`, `Overdue Reviews`, `Ending Soon (30d)` (all `0` on the crawl). Card: `Employees On Probation`.

- **Data grid** — Columns: `Employee`, `Branch`, `Start`, `End`, `Reviews`, `Next Review`, `Days Left`, `Action`. One row = one employee's active probation with its review progress. Empty-state row: `No one on probation.`
- **Connections** — `Start Probation` places an `/employees` record onto a probation defined by a `/hrms/probation-templates` template; outcomes are analysed on `/hrms/probation-report`. The final confirm/terminate decision maps to the `Probation Decision` approval type in `/approval/config` and is actioned via `/approvals` (the report has an `Awaiting Approval` KPI).
- **Automation notes** — `Start Probation` is a button (not a link) — expect a modal/form to pick employee + template. `Report` and `Templates` are plain anchors (`href="/hrms/probation-report"`, `href="/hrms/probation-templates"`), good for navigation tests. KPI tiles render numeric values — assert `0` on a clean tenant, and increments after starting a probation.

Probation Templates (`/hrms/probation-templates`)

- **Purpose** — Define reusable probation templates: duration, checkpoint review days, evaluation criteria, and which template is the default.
- **UI elements** — Button: `New Template`.
- **Data grid** — Columns: `Name`, `Duration`, `Checkpoints (days)`, `Criteria`, `Default`, `Active`, `Action`. One row = one probation template. Empty-state row: `No templates yet.`
- **Connections** — Templates are consumed by `Start Probation` on `/hrms/probation`; checkpoint days drive the `Reviews`/`Next Review` columns and the review-due KPIs there.
- **Automation notes** — `New Template` should open a create modal/form (no inline form inputs were captured on the list page). `Default` and `Active` columns imply toggle behaviors worth testing (only one default at a time; inactive templates hidden from Start Probation). Empty state is a table row with text `No templates yet.`

Probation Report (`/hrms/probation-report`)

- **Purpose** — Analytical report over probations started in a date range: outcome KPIs, completion rate, and a detail grid, exportable to Excel.
- **UI elements** — Filters: `From (start date)` (date input), `To (start date)` (date input), `Branch` select (default `All branches`). Buttons: `Run Report`, `Export Excel`. KPI tiles: `Total`, `On Probation`, `Confirmed`, `Terminated`, `Awaiting Approval`, `Overdue Reviews`, `Completion Rate` (shown as `0%`). Card: `Details (0)` — the count in the title tracks result size.
- **Data grid** — Columns: `Employee`, `Branch`, `Start`, `End`, `Outcome`, `Reviews`, `Overdue`, `Decision`. One row = one probation record in range with its outcome and decision. Empty-state row: `No probations in this range.`
- **Connections** — Aggregates the probations run from `/hrms/probation` (using `/hrms/probation-templates` definitions); `Awaiting Approval` and `Decision` tie back to the `Probation Decision` workflow in `/approval/config` / `/approvals`.
- **Automation notes** — Report is pull-based: set filters then click `Run Report` (grid says `No probations in this range.` until then). `Export Excel` triggers a file download — use Playwright's download event. The `Details (0)` card title doubles as a row-count assertion target.

Resigned Employees (`/resigned-employees`)

- **Purpose** — Read-only register of employees who have resigned/left, with their exit date and contact details.

- **UI elements** — Name filter text input (`input#filter-name`). No action buttons captured. Card: `Resigned Employees`.
- **Data grid** — Columns: `SlNo`, `Date`, `Name`, `Phone`, `Designation`, `Nationality`. One row = one resigned employee. (Crawl captured one blank row — treat empty-string rows as no data.)
- **Connections** — Populated when an employee from `/employees` is marked resigned/archived (see the `Include archived` toggle and `Status` column there). Resigned employees drop out of `/worker-directory` and active payroll in `/employee-salary-process`.
- **Automation notes** — No create/edit actions on this page — resignations are triggered from the employee record, so tests must set up data via `/employees`. `#filter-name` is the only interactive control; verify it filters the grid client-side. The empty grid rendered a row with empty cells rather than an explicit empty-state message.

Employee Excel Import (`/upload-employee`)

- **Purpose** — Bulk onboarding: upload an Excel file of employees using the provided sample format.
- **UI elements** — File input (`input[type=file]`) labelled `Upload Recipient Excel File`; link `Download Sample Excel` (`href="/assets/images/EmployeeExcelImport.xlsx"`); buttons `Excel Rules`, `Upload`.
- **Data grid** — None.
- **Connections** — Successful imports create records in `/employees`, which then flow to `/worker-directory`, payroll and the rest of Core HR.
- **Automation notes** — Copy bug: the page card is titled `Leave Request` although the page is the employee import — do not assert on that card title. `Excel Rules` likely opens an info modal describing the required columns. Test flow: download `/assets/images/EmployeeExcelImport.xlsx`, fill it, `setInputFiles` on the file input, click `Upload`, then verify the rows appear in `/employees`. Include a negative test with a malformed sheet.

Letter Templates (`/letters/templates`)

- **Purpose** — Manage reusable HR letter templates (offer letters, warnings, etc.) that are later merged with employee data.
- **UI elements** — Buttons/links: `Merge Fields` (link to `/letters/fields`), `Generate Letter` (link to `/letters/generate`), `New Template` button.
- **Data grid** — Columns: `Name`, `Owner`, `Type`, `Subject`, `Active`, `Action`. One row = one letter template. Empty-state row: `No templates yet.`
- **Connections** — Templates authored here (with placeholders from `/letters/fields`) are the `Template` options on `/letters/generate`.
- **Automation notes** — `New Template` opens the template editor (modal or route — no inline inputs captured). `Merge Fields` and `Generate Letter` are anchors with stable hrefs for navigation tests. `Active` column implies an enable/disable toggle affecting availability in the generate page.

Generate Letter (`/letters/generate`)

- **Purpose** — Produce a letter for a specific employee from a template: preview the merged output, generate it, and optionally email it.
- **UI elements** — Selects: `Template` (`-- Select template --`), `Branch` (`-- Select branch --`), `Employee` (`-- Select employee --`). Checkbox `Email the letter` (`input#sendmail`). Buttons: `Preview`, `Generate`. Card: `Preview` with hint text `Select a template and employee, then Preview.`

- **Data grid** — None.
- **Connections** — Consumes templates from `/letters/templates` (and merge fields from `/letters/fields`); employee list from `/employees` (branch-filtered). Breadcrumb is `Letter Templates  Generate`, confirming it is a child of the templates page.
- **Automation notes** — Template + employee are effectively mandatory (preview hint enforces it); Branch likely filters the Employee select. `#sendmail` is a stable selector for the email toggle. Test sequence: select template → branch → employee → `Preview` (assert preview card fills) → `Generate`. On the empty tenant the Template select has no options — seed a template first via `/letters/templates`.

My Approvals (`/approvals`)

- **Purpose** — The approver's workbench: items awaiting the signed-in user's decision, the user's own submitted requests, and a decision history.
- **UI elements** — Tabs: `Awaiting my decision (0)`, `My requests`, `History`. Button: `Refresh`. Card: `Awaiting My Decision (0)`.
- **Data grid** — Columns: `Type`, `Details`, `Level`, `As`, `Raised`, `Action`. One row = one pending approval item (its document type, chain level, and the role the user approves as). Empty-state row: `Nothing awaiting your approval.`
- **Connections** — Receives documents routed by the workflows defined in `/approval/config`. Within Core HR that includes `Salary Revision` items raised on `/salary-revisions` and `Probation Decision` items from `/hrms/probation`; approving updates the `Status`/`Decision` shown on those pages and on `/hrms/probation-report`.
- **Automation notes** — Tab labels embed live counts (`Awaiting my decision (0)`) — match with a regex, not exact text. Breadcrumb group is `Workflow`, not `HRMS`. The `Action` column will carry approve/reject controls once rows exist; multi-user tests are needed (raiser vs approver accounts). `Refresh` re-polls without navigation.

Approval Configuration (`/approval/config`)

- **Purpose** — Define approval workflows: pick a document type, name the workflow, add ordered approval levels, and optionally mark it as the default for that type.
- **UI elements** — Card `New Workflow`: `Approval Type` select (`-- Select type --`) with options `Salary Revision`, `Leave Request`, `Expense Claim`, `Policy Publish`, `Job Requisition`, `Offer`, `Compensation Cycle`, `Document Verification`, `Benefit Enrollment`, `Attendance Regularization`, `Overtime`, `Material Purchase Request`, `Stock Transfer`, `Supplier PO`, `Sales Order`, `Quotation`, `Credit Limit Override`, `Sales Return`, `Profile Change Request`, `Com Off Request`, `Encashment Request`, `Geo Fence Request`, `Probation Decision`; `Workflow Name` text input (placeholder `e.g. Long Leave Approval`); checkbox `Default for this type` (`input#cfgDefault`); `+ Add level` button under `Approval Levels`; `Save Workflow` button. Card `Configured Workflows` holds the grid. Helper text: `No levels = direct (auto-approve, no approval step).`
- **Data grid** — Columns: `Type`, `Name`, `Approval chain`. One row = one configured workflow. Empty-state row: `No workflows configured yet.`
- **Connections** — Workflows configured here drive routing into `/approvals` for every listed type; within Core HR, `Salary Revision` governs `/salary-revisions` statuses and `Probation Decision` governs probation outcomes on `/hrms/probation` / `/hrms/probation-report`. Multiple named workflows per type are allowed; the default applies when a document doesn't pick a specific one.

- **Automation notes** — `+ Add level` appends approval-level rows dynamically — test 0-level (auto-approve), 1-level and multi-level chains. `#cfgDefault` is a stable checkbox selector. Key behavioral assertion from the page's own copy: with no levels the document auto-approves (no `/approvals` entry). Breadcrumb group is `Workflow`.

Employee Deduction Reports (`/employee-deduction-report`)

- **Purpose** — Filterable report over deduction entries by deduction type, staff member and time period.
- **UI elements** — Filters: `Deduction Type` select (default `All`, `select#report-assignee`), `Staff` select (default `All`, also captured as `select#report-assignee` — duplicate id), `Period` select (`select#timePeriodSelect`; options `Choose`, `This Week`, `This Month`, `This Year`, `Custom`). Button: `View Report`.
- **Data grid** — None rendered until `View Report` is run (no table captured on load).
- **Connections** — Reports over entries created on `/employee-deduction`. Breadcrumb group: `HRMS Reports`.
- **Automation notes** — Pull-based report: results only render after `View Report`. Duplicate `#report-assignee` id on both Deduction Type and Staff selects — locate by label or position, never by id alone. `Period = Custom` presumably reveals date inputs (not present in the initial capture) — verify dynamically. Seed a deduction via `/employee-deduction` before asserting report rows.

Employee Remark Reports (`/employee-remark-report`)

- **Purpose** — Filterable report over remark entries by staff member and time period.
- **UI elements** — Filters: `Staff` select (default `All`, `select#report-assignee`), `Period` select (`select#timePeriodSelect`; options `Choose`, `This Week`, `This Month`, `This Year`, `Custom`). Button: `View Report`.
- **Data grid** — None rendered until `View Report` is run (no table captured on load).
- **Connections** — Reports over entries created on `/employee-remark`. Breadcrumb group: `HRMS Reports`.
- **Automation notes** — Same pull-based pattern and same id scheme as the deduction report (`#report-assignee`, `#timePeriodSelect`) — shared page object is practical. Remember the source entry page `/employee-remark` carries the "Employee Deduction" title bug when writing end-to-end assertions. Test `Custom` period for dynamically revealed date range inputs.

Process flows

- **Employee onboarding**: define masters in `/sections` (department → section) → create the employee via `/employees` (`New Employee`) or bulk-load via `/upload-employee` (download sample → fill → `Upload`) → record becomes visible in `/employees` grid and `/worker-directory` (Cards / Org Chart) → optionally `Start Probation` on `/hrms/probation`.
- **Probation lifecycle**: create a template on `/hrms/probation-templates` (duration, checkpoint days, criteria, default) → `Start Probation` on `/hrms/probation` for an employee → checkpoint reviews tracked on the dashboard (`Reviews`, `Next Review`, `Days Left`, due/overdue KPIs) → final confirm/terminate goes through the `Probation Decision` workflow (`/approval/config` → `/approvals`) → outcomes and completion rate analysed on `/hrms/probation-report` (`Run Report`, `Export Excel`).
- **Salary revision**: configure a `Salary Revision` workflow in `/approval/config` (0 levels = auto-approve) → raise the revision on `/salary-revisions` (branch → employee → new gross → effective date → reason) → item appears in the approver's `/approvals` inbox (`Awaiting my decision`) →

decision updates the `Status` in the `Revision History` grid → the new gross feeds `Basic Salary` in `/employee-salary-process`.

- **Monthly payroll**: record any deductions in `/employee-deduction` → on `/employee-salary-process` pick Branch + `Salary Year` + `Salary Month` → review `Staff Name / Basic Salary / No of Leave / Payable Amount` grid → `Save` the run.
- **Deduction reporting**: `/employee-deduction` (Save entry) → `/employee-deduction-report` (Deduction Type / Staff / Period → `View Report`).
- **Remark reporting**: `/employee-remark` (Save entry) → `/employee-remark-report` (Staff / Period → `View Report`).
- **HR correspondence**: author a template on `/letters/templates` (`New Template`, placeholders from `Merge Fields` at `/letters/fields`) → `/letters/generate`: select Template + Branch + Employee → `Preview` → `Generate` (optionally `Email the letter`).
- **Separation**: employee is resigned/archived from their `/employees` record (`Status`, `Include archived` toggle) → appears in the `/resigned-employees` register → excluded from `/worker-directory` results and active payroll.

Deep Dive — Recruitment & Onboarding

Overview

The Recruitment & Onboarding sub-module of the Progbiz HRMS ERP (https://hrms-erp.progbiz.in) covers the full hire-to-onboard lifecycle. It starts with internal manpower demand (`/requisition-list`), turns approved demand into published job openings (`/vacancy-list`), and exposes those openings on a public careers page (`/current-openings`, "Join Our Team"). Inbound applicants surface in `/job-applications-list` and are managed as candidates in `/candidates`, where they move through follow-up statuses (New → In Progress → Shortlisted → Selected / Rejected). Screening is supported by an assessment library (`/assessment-list`), scheduled interviews (`/interview-schedules`), and a per-vacancy Kanban board (`/recruitment-pipeline`) with stage configuration and scoring. Successful candidates receive offers in `/offer-list`; unsuccessful or parked candidates are archived and searchable in `/talent-pool`. Master-data pages — `/candidate-status` (followup statuses) and `/interview-rounds` (round names and ordering) — feed the candidate and interview screens, while `/communication-templates` holds reusable candidate emails/messages. Once an offer is accepted, onboarding is driven by checklists defined in `/onboarding-templates` and executed per new hire in `/onboarding-pipeline`.

Page Index

#	Page	Route	Purpose
1	Job Requisitions	`/requisition-list`	Raise and track manpower requests through an approval workflow
2	Hiring — Job Openings	`/vacancy-list`	Create, publish and manage job openings (vacancies)
3	Current Openings (public careers)	`/current-openings`	Public "Join Our Team" page where applicants browse openings and apply
4	Job Applications	`/job-applications-list`	Review inbound applicants; schedule interview or reject
5	Candidates	`/candidates`	Central candidate register with status-bucket tabs
6	Assessments	`/assessment-list`	Library of reusable assessments (type, max score, attachment)
7	Interview Schedule	`/interview-schedules`	Schedule and track interviews per candidate/round
8	Offer Management	`/offer-list`	Create and track offers (CTC, joining date, status)
9	Recruitment Pipeline (Kanban)	`/recruitment-pipeline`	Per-vacancy Kanban board with configurable stages and scoring
10	Communication Templates	`/communication-templates`	Reusable candidate communication templates (name, type, subject)
11	Talent Pool & Archive	`/talent-pool`	Search archived/active candidates by name, skill and pool
12	Candidate Followup Status (Settings)	`/candidate-status`	Master list of candidate followup statuses and their nature

#	Page	Route	Purpose
13	Interview Rounds (Settings)	`/interview-rounds`	Master list of interview rounds and their order
14	Onboarding Templates	`/onboarding-templates`	Define reusable onboarding checklists/templates
15	Onboarding Pipeline	`/onboarding-pipeline`	Start and track active onboardings for new hires

Pages

Job Requisitions (`/requisition-list`)

- **Purpose** — Entry point of the hiring process: departments raise manpower requests ("Manpower Requests" card) for a designation/branch, which travel through an approval workflow before becoming vacancies.
- **UI elements**
  - Button: `New Requisition` (opens the create-requisition form/modal).
  - Status filter select with options (from body text): `All Status`, `Draft`, `Submitted`, `Approved`, `Rejected`, `Returned`.
  - Text input with placeholder `Search designation...`.
  - Breadcrumb: "Job Requisitions › Recruitment › Requisitions".
- **Data grid** — Columns: `Sl.No`, `Designation`, `Department`, `Positions`, `Type`, `Work Type`, `Status`, `Branch`, `Action`. One row = one manpower request (a demand for N positions of a designation in a department/branch). Captured with 0 rows.
- **Connections** — Approved requisitions logically feed `/vacancy-list` (a requisition for a designation becomes a job opening). Status values (`Draft`/`Submitted`/`Approved`/`Rejected`/`Returned`) imply a submit-and-approve chain that gates vacancy creation.
- **Automation notes** — Table rendered empty (rowCount 0) on capture: tests must handle the no-rows state. Filter by status via the single native `<select>`; search via the `Search designation...` placeholder input. `New Requisition` is the only primary action button — expect a modal/form on click. Assert row counts after switching the status filter.

Hiring — Job Openings (`/vacancy-list`)

- **Purpose** — The "Hiring" workspace for recruiters: manage job openings (vacancies), see candidate counts per opening, and control publishing state.
- **UI elements**
  - Tabs: `Job Openings`, `Candidates`, `Talent Pools` (in-page tab strip switching between hiring views).
  - Button: `Add Job Opening`.
  - Status filter select with options (from body text): `All`, `Open`, `Draft`, `On Hold`, `Filled`, `Cancelled`.
  - Summary/counter text: "0 published · 0 total · Show Draft & Open" — publishing counters plus a display toggle.
- **Data grid** — Columns: `Candidates`, `Job Opening`, `Hiring Lead`, `Created On`, `Status`, `Action`. One row = one vacancy/job opening with its owning hiring lead and candidate count. Captured with 0 rows.

- **Connections** — Fed by approved requisitions from `/requisition-list`. "Published" openings surface on the public `/current-openings` page. The `Candidates` tab connects to the `/candidates` register and the `Talent Pools` tab to `/talent-pool`. Each opening is also the vacancy selected on `/recruitment-pipeline`.
- **Automation notes** — The tab strip (`Job Openings` / `Candidates` / `Talent Pools`) switches content without route change — scope selectors to the active tab panel. "0 published · 0 total" text is a useful assertion target after publish/unpublish actions. `Add Job Opening` should open a create form/modal. Single native `<select>` drives the status filter; empty table state must be handled.

Current Openings — public careers page (`/current-openings`)

- **Purpose** — Public-facing "Join Our Team" page where external applicants pick a published opening and apply. This is the applicant-side mirror of `/vacancy-list`.
- **UI elements**
  - Heading: `Join Our Team`.
  - A single select with id `selectbox` (default option `Choose`) listing published openings, next to a blue search/submit icon button (visible in screenshot).
  - No breadcrumb, no sidebar, no ERP shell — this page renders outside the authenticated app chrome.
- **Data grid** — None. Page is a picker; with no published vacancies it renders only the heading and empty select.
- **Connections** — Populated by openings published from `/vacancy-list`. Submitted applications land in `/job-applications-list` (whose rows carry `Position Applied For`).
- **Automation notes** — Use `#selectbox` as the selector for the opening picker. The page is public (no ERP navigation/breadcrumb captured) — tests can hit it without the logged-in storage state, but must seed a *published* opening first or the select stays on `Choose` with no options. The apply form only appears after choosing an opening, so end-to-end tests need `select → search button → form` sequencing.

Job Applications (`/job-applications-list`)

- **Purpose** — Inbox of inbound applicants ("Applicant Details" card): review each application and either move it forward (schedule an interview) or reject it.
- **UI elements**
  - No buttons, tabs, filters or inputs were captured at page level — all actions live in the table's per-row action columns.
  - Breadcrumb/header: "Job Applications".
- **Data grid** — Columns: `Sl.No`, `Name`, `Position Applied For`, `Phone`, `Email`, `Details`, `Status`, `Schedule Interview`, `Reject`. One row = one application submitted for a specific position. Captured with 0 rows.
- **Connections** — Fed by applications from `/current-openings`. The `Schedule Interview` column action feeds `/interview-schedules`; progressed applicants appear as candidates in `/candidates`; rejected applicants logically flow to the archive in `/talent-pool`.
- **Automation notes** — Row-level actions (`Details`, `Schedule Interview`, `Reject`) are table columns, not toolbar buttons — target them per-row (e.g. `row.getByRole('button')` within the matching cell). Expect a confirmation or scheduling modal on `Schedule Interview` / `Reject`. Empty state must be handled; seeding requires an application submitted via the public page or API.

Candidates (`/candidates`)

- **Purpose** — Central candidate register for the whole funnel, bucketed by followup status with per-bucket counts.
- **UI elements**
  - Button: `Add New` (manual candidate entry; the captured `Candidate Name` and `Phone Number` placeholder inputs belong to this add form/modal).
  - Status tabs rendered as buttons with live counts: `New 0`, `In Progress 0`, `Shortlisted 0`, `Selected 0`, `Rejected 0`.
  - Text input with placeholder `Search Name/Phone` (list search).
  - Form inputs captured: `Candidate Name`, `Phone Number` text inputs plus four unlabeled selects (branch/designation/status-type pickers on the add form and list filters).
- **Data grid** — Columns: `Sl.No`, `Name`, `Email`, `Phone`, `Branch`, `Designation`, `Skills`, `Status`, `Added On`, `Action`. One row = one candidate profile. Captured rowCount 1 with an empty first row (placeholder/empty-state row).
- **Connections** — Receives candidates from `/job-applications-list` and manual `Add New` entry; also reachable via the `Candidates` tab on `/vacancy-list`. `Status` values are governed by the `/candidate-status` master. Candidates flow onward to `/interview-schedules`, `/recruitment-pipeline`, `/assessment-list` usage, and — when `Selected` — to `/offer-list`. Archived/rejected candidates surface in `/talent-pool`.
- **Automation notes** — The status buckets (`New`/`In Progress`/`Shortlisted`/`Selected`/`Rejected`) are buttons, not links — click them and assert the count suffix in the button label (e.g. `Shortlisted 3`). The `Add New` flow exposes `Candidate Name` / `Phone Number` placeholders as stable selectors. Distinguish the list-filter selects from the add-form selects by scoping to the modal when it is open. The grid can render a single empty row — do not treat rowCount 1 as data present; assert on cell text.

Assessments (`/assessment-list`)

- **Purpose** — "Assessment Library": reusable assessment definitions (tests/tasks) with a type, description, maximum score and optional attachment, used to evaluate candidates.
- **UI elements**
  - Button: `New Assessment`.
  - Breadcrumb: "Assessments › Recruitment › Assessments".
- **Data grid** — Columns: `Sl.No`, `Title`, `Type`, `Description`, `Max Score`, `Attachment`, `Action`. One row = one assessment definition. Captured first row showed `Loading...` (async fetch in progress).
- **Connections** — Assessments are consumed during candidate evaluation — logically feeding the `Score` action on `/recruitment-pipeline` and interview evaluation in `/interview-schedules`; scores ultimately show as the `Score` column in `/talent-pool`.
- **Automation notes** — The grid initially renders a `Loading...` row: tests must wait for that text to disappear (`expect(row).not.toHaveText('Loading...')`) before asserting data. `New Assessment` should open a create modal/form including a file attachment field (implied by the `Attachment` column) — plan for Playwright `setInputFiles`.

Interview Schedule (`/interview-schedules`)

- **Purpose** — "Scheduled Interviews": create and track interview appointments per candidate, round, date/time and mode.
- **UI elements**

    - Button: `Schedule Interview` (opens scheduling form/modal).
    - Breadcrumb: "Interviews › Recruitment › Interview Schedule".
- **Data grid** — Columns: `Sl.No`, `Candidate`, `Round`, `Scheduled On`, `Mode`, `Status`, `Action`. One row = one scheduled interview for a candidate in a specific round. Captured with 0 rows.
- **Connections** — Reached from the `Schedule Interview` action on `/job-applications-list` and used for candidates from `/candidates`. The `Round` values come from the `/interview-rounds` master. Interview outcomes drive candidate status changes (`/candidate-status` values on `/candidates`) and stage moves on `/recruitment-pipeline`; passed candidates progress to `/offer-list`.
- **Automation notes** — Single primary button `Schedule Interview` → expect a modal with candidate, round, date and mode fields (Round options depend on `/interview-rounds` seed data). Empty grid on capture — handle no-rows state; after scheduling, assert a row with the chosen candidate and `Scheduled On` value.

Offer Management (`/offer-list`)

- **Purpose** — "Offers": issue and track job offers with total CTC, joining date and offer status.
- **UI elements**
    - Button: `New Offer`.
    - Breadcrumb: "Offers › Recruitment › Offer Management".
- **Data grid** — Columns: `Sl.No`, `Candidate`, `Total CTC`, `Joining`, `Status`, `Action`. One row = one offer extended to a candidate. Captured with 0 rows.
- **Connections** — Fed by `Selected` candidates from `/candidates` (after interviews in `/interview-schedules`). An accepted offer with its `Joining` date is the logical trigger for `Start Onboarding` on `/onboarding-pipeline` (using a checklist from `/onboarding-templates`). Offer letters/communications draw on `/communication-templates`.
- **Automation notes** — `New Offer` opens the offer form (candidate picker + CTC + joining date expected). Empty state must be handled. `Total CTC` and `Joining` cells are good assertion targets after creation; `Status` transitions (offer accepted/declined) are the hook for onboarding tests.

Recruitment Pipeline — Kanban (`/recruitment-pipeline`)

- **Purpose** — Kanban view of a vacancy's candidate funnel: pick a vacancy, see candidates as cards in configurable stage columns, score them and keep stages in sync.
- **UI elements**
    - `Vacancy` select with options `— Select a vacancy —` and `All vacancies` (plus one option per open vacancy).
    - Buttons: `Configure Stages` (top-right, opens stage configuration) and `Score` (rendered muted/inactive until a vacancy with candidates is selected — screenshot shows it greyed next to the vacancy picker with a refresh icon button beside it).
    - `Auto-Sync` toggle — checkbox with id `autoSync`.
- **Data grid** — None; this is a Kanban board. With no vacancy selected the board area is empty (screenshot confirms only the filter bar renders).
- **Connections** — Vacancies come from `/vacancy-list`; cards are candidates from `/candidates`. `Score` ties to the `/assessment-list` library and to the `Score` column in `/talent-pool`. Stage movement mirrors candidate followup statuses (`/candidate-status`) — the `Auto-Sync` toggle suggests automatic status↔stage synchronisation. End of pipeline feeds `/offer-list`.

- **Automation notes** — Mandatory filter: nothing renders until a vacancy is chosen in the `Vacancy` select — every pipeline test must first select a vacancy (or `All vacancies`). Use `#autoSync` for the toggle. `Configure Stages` opens a stage-editing UI — treat as modal/panel. Kanban drag-and-drop (stage moves) will need Playwright `dragTo`/manual mouse events; there are no table selectors, so target cards by candidate name text.

Communication Templates (`/communication-templates`)

- **Purpose** — "Candidate Communication" template library: reusable messages (e.g. emails) with a name, type and subject for candidate outreach at each stage.
- **UI elements**
  - Button: `New Template`.
  - Breadcrumb: "Candidate Communication › Recruitment › Communication Templates".
- **Data grid** — Columns: `Sl.No`, `Name`, `Type`, `Subject`, `Action`. One row = one communication template. Captured first row showed `Loading...` (async fetch).
- **Connections** — Supporting/master page: templates are logically consumed when communicating with applicants and candidates from `/job-applications-list`, `/candidates`, `/interview-schedules` (invites) and `/offer-list` (offer letters).
- **Automation notes** — Wait out the `Loading...` first row before asserting. `New Template` should open a create form with name/type/subject and body fields — a rich-text editor is likely, so prefer `data-testid`/role-based selectors over CSS classes for the body area.

Talent Pool & Archive (`/talent-pool`)

- **Purpose** — Searchable archive of candidates: query by name/email, skill and pool (archived vs active) to resurface past candidates for new openings.
- **UI elements**
  - Button: `Search` (explicit submit — results load on click, not on keystroke).
  - Text input with placeholder `Name or email`.
  - `Skill` select (default `— Any —`).
  - `Pool` select with options (from body text): `Archived (talent pool)`, `Active`, `All`.
- **Data grid** — Columns: `Sl.No`, `Name`, `Designation`, `Skills`, `Tags`, `Status`, `Score`, `Action`. One row = one pooled/archived candidate with skill tags and evaluation score. Captured first row showed `Loading...`.
- **Connections** — Fed by rejected/parked candidates from `/candidates` and `/job-applications-list`; also reachable via the `Talent Pools` tab on `/vacancy-list`. The `Score` column reflects assessment/pipeline scoring (`/assessment-list`, `/recruitment-pipeline`). Pool candidates can be re-engaged into new vacancies from `/vacancy-list`.
- **Automation notes** — Search is button-driven: fill `Name or email`, choose `Skill`/`Pool`, then click `Search` and wait for the grid refresh (initial `Loading...` row must clear). The `Pool` select defaults to the archived view — tests asserting active candidates must switch it. Two native selects + one text input make stable placeholder/option-text selectors.

Candidate Followup Status — Settings (`/candidate-status`)

- **Purpose** — HRMS Settings master ("Status List"): define candidate followup statuses and their nature, which drive the candidate lifecycle buckets.
- **UI elements**

    - Button: `Add Followup Status`.
    - Breadcrumb: "Candidate Status › HRMS › Settings › Candidate Followup Status".
- **Data grid** — Columns: `Sl No`, `Status Name`, `Nature`, `Action`. One row = one followup status definition (its `Nature` maps it to a lifecycle bucket such as new/in-progress/positive/negative). Captured first row showed `Loading...`.
- **Connections** — Master data consumed by `/candidates` (the `Status` column and the New/In Progress/Shortlisted/Selected/Rejected buckets) and logically by `/recruitment-pipeline` stage sync.
- **Automation notes** — Note the header spelling `Sl No` (no dot) vs `Sl.No` elsewhere — keep column selectors page-specific. Wait for `Loading...` to clear. `Add Followup Status` opens a small create modal (name + nature). As shared master data, mutations here affect `/candidates` tests — prefer creating uniquely-named statuses and cleaning up via the `Action` column.

Interview Rounds — Settings (`/interview-rounds`)

- **Purpose** — HRMS Settings master: define interview round names and their sequence (`Order`) used when scheduling interviews.
- **UI elements**
    - Button: `Add Round`.
    - Breadcrumb: "Interview Rounds › HRMS › Settings › Interview Rounds".
- **Data grid** — Columns: `Sl No`, `Round Name`, `Order`, `Action`. One row = one interview round definition. Captured first row showed `Loading...`.
- **Connections** — Master data for the `Round` column/picker in `/interview-schedules`; round ordering shapes the interview progression that ultimately feeds `/offer-list`.
- **Automation notes** — Same `Sl No` header spelling and `Loading...` async-row behaviour as `/candidate-status`. `Add Round` opens a create modal (round name + order). Seed rounds before any `/interview-schedules` test, since scheduling needs at least one round to exist.

Onboarding Templates (`/onboarding-templates`)

- **Purpose** — Define reusable onboarding checklists ("Templates") that structure a new hire's onboarding tasks.
- **UI elements**
    - Button: `New Template`.
    - "Templates" card acting as a template list panel; empty-state texts: "No templates yet." and "Select a template or create a new one." — a master/detail layout (template list on the left, detail/editor on the right).
    - Breadcrumb: "Onboarding Templates › HRMS › Onboarding Templates".
- **Data grid** — None (list/detail panels rather than a table).
- **Connections** — Templates are consumed by `/onboarding-pipeline` when starting an onboarding for a hired candidate (post-`/offer-list`).
- **Automation notes** — Assert the dual empty-state strings ("No templates yet." / "Select a template or create a new one.") before creation. `New Template` likely opens an inline editor in the detail pane rather than a modal — after creating, select the template in the list panel and assert the detail pane populates. No table selectors exist; target list items by template name text.

Onboarding Pipeline (`/onboarding-pipeline`)

- **Purpose** — Execute and track active onboardings: each started onboarding runs a new hire through the checklist stages defined by an onboarding template.
- **UI elements**
  - Button: `Start Onboarding` (top-right, primary).
  - Empty state text: "No active onboardings." (screenshot confirms an otherwise blank canvas).
  - Breadcrumb: "Onboarding Pipeline › HRMS › Onboarding Pipeline".
- **Data grid** — None captured; with zero active onboardings the page shows only the empty-state message.
- **Connections** — The terminus of the recruitment flow: candidates with accepted offers in `/offer-list` are started here, using a checklist from `/onboarding-templates`. Completed onboarding logically hands the person over to core HRMS employee records (outside this sub-module).
- **Automation notes** — Assert the exact empty state "No active onboardings." as the baseline. `Start Onboarding` should open a modal/wizard (candidate + template selection) — it requires both an offer-stage candidate and at least one onboarding template as preconditions, so seed `/offer-list` and `/onboarding-templates` first. Board/card layout after starting is unverified in captures — discover selectors at test time.

Process flows

- **Demand → Opening → Publication**: `/requisition-list` (raise `New Requisition` → status `Draft` → `Submitted` → `Approved`) → `/vacancy-list` (`Add Job Opening`, status `Open`, counted in "published · total") → published opening appears in the `#selectbox` on public `/current-openings`.
- **Application intake**: applicant applies on `/current-openings` → row appears in `/job-applications-list` (`Position Applied For` links it to the opening) → per-row `Schedule Interview` (→ `/interview-schedules`) or `Reject` (→ archive in `/talent-pool`).
- **Candidate progression**: candidate exists in `/candidates` (via application intake or `Add New`) → moves through followup buckets `New` → `In Progress` → `Shortlisted` → `Selected`/`Rejected`, with the bucket vocabulary defined in `/candidate-status`.
- **Evaluation loop**: `/interview-schedules` rounds (round names/order from `/interview-rounds`) + assessments from `/assessment-list` → scores and stage moves visualised per vacancy on `/recruitment-pipeline` (`Configure Stages`, `Score`, `Auto-Sync` keeping stages and candidate statuses aligned).
- **Offer → Onboarding**: `Selected` candidate → `/offer-list` `New Offer` (Total CTC + Joining date) → on acceptance → `/onboarding-pipeline` `Start Onboarding` using a checklist from `/onboarding-templates` → completed hire exits the recruitment sub-module into core HRMS.
- **Recycling talent**: rejected/parked candidates accumulate in `/talent-pool` (filter `Archived (talent pool)` / `Active` / `All`, search by skill) and can be re-engaged against new openings from `/vacancy-list`.
- **Communication throughout**: templates from `/communication-templates` (name/type/subject) back candidate messaging at every stage — application acknowledgements, interview invites and offer letters.

Deep Dive — Attendance & Time

Overview

The Attendance & Time sub-module of the Progbiz HRMS ERP (https://hrms-erp.progbiz.in) covers everything from shift definition to month-end attendance lock. Administrators define shift masters on `/shifts` and map them to the organisation on `/shift-roster` using a scope hierarchy (Company → Branch → Department → Employee, most-specific wins). Raw punch data arrives from biometric devices (`/data-from-device`) and from mobile field check-ins (`/add-visit-report`), with geofence locations (`/geofences`) governing where mobile punches are valid. The processed day-level attendance is viewed on `/attendance-log`, corrected through `/regularization`, and pushed through approval queues: worked-day approval (`/approval-operation` → `/approval-operation-report`), absent-day approval (`/approval-absent` → `/approval-absent-report`) and overtime approval (`/overtime-approval`). `/timesheet` reconciles attendance hours against task hours, `/attendance-report-pack` produces an exportable Daily Register, and `/attendance-finalization` runs the pay-cycle cut-off that locks the period for payroll. For QA purposes the sub-module splits into two UI generations: newer "Attendance"-breadcrumb pages with inline filter bars and card layouts, and older "HRMS"-breadcrumb report pages whose grids stay empty until a `Filter` dialog is applied.

Page Index

#	Page	Route	Purpose
1	Shifts	`/shifts`	Define shift masters (name, type, timing, night flag)
2	Shift Roster	`/shift-roster`	Assign shifts to company/branch/department/employee with effective dates
3	Attendance Log	`/attendance-log`	Day-wise processed attendance report per employee
4	Data from device	`/data-from-device`	Raw biometric/device punch records with sync details
5	Add Visit Report	`/add-visit-report`	Mobile site-visit check-in/check-out log with location evidence
6	Regularization	`/regularization`	Raise and track attendance correction requests
7	Overtime Approval	`/overtime-approval`	Approve overtime minutes and mark payout/export status
8	Attendance Finalization	`/attendance-finalization`	Start/refresh and finalize a pay-cycle attendance run
9	Geofence Locations	`/geofences`	Maintain geofence points (lat/long/radius) for mobile punching
10	Timesheet	`/timesheet`	Compare daily attendance hours vs task hours per employee
11	Attendance Report Pack	`/attendance-report-pack`	Exportable Daily Register report with pagination
12	Approval Operation	`/approval-operation`	Approve worked-day attendance records (late/early/OT detail)
13	Approval Operation Report	`/approval-operation-report`	Review/delete already-approved worked-day records

#	Page	Route	Purpose
14	Approval Absent	`/approval-absent`	Approve absent-day records
15	Approval Absent Report	`/approval-absent-report`	Review/delete already-approved absent-day records

Pages

Shifts (`/shifts`)

- **Purpose** — Master screen for shift definitions: the catalogue of shifts (name, type, timing, night-shift flag, active state) that every other attendance calculation resolves against.
- **UI elements** — `New Shift` button (opens the create-shift form/modal); card titled `Shift Definitions`; text input with placeholder `Search by shift name`; two unlabeled single-select filters (type/active-style filters).
- **Data grid** — Columns: `Sl.No`, `Shift Name`, `Type`, `Timing`, `Night`, `Active`, `Action`. One row = one shift definition; `Action` holds per-row edit/manage controls.
- **Connections** — Feeds `/shift-roster` (a roster assignment picks a shift from this list); shift name surfaces again in `/overtime-approval` (`Shift` column) and `/timesheet` (`Shift` column). Breadcrumb group: Attendance.
- **Automation notes** — Capture shows `rowCount: 1` with an empty first row, i.e. the tbody renders a single empty/placeholder row when no data matches — assert on cell text, not row count. `Search by shift name` placeholder is a stable locator (`getByPlaceholder`). `New Shift` should open a creation dialog — test both create and the `Active` toggle round-trip.

Shift Roster (`/shift-roster`)

- **Purpose** — Assigns a shift to an organisational scope for a date range. On-page help text states: "Pick the branch, department or employee for this assignment. The most-specific assignment wins when a shift is resolved. Sub-departments inherit the parent department's shift unless they have an assignment of their own."
- **UI elements** — Two cards: `Assign A Shift` (inline form) and `Current Assignments` (grid). Form fields: `Shift*` (select, default `-- select --`), `Scope*` (select with `Company` / `Branch` / `Department` / `Employee`), `Target` (text/lookup), `Effective From*` (date), `Effective To` (date), and the `Assign` submit button. Grid-side filters: text input `Search scope or shift...`, two selects, and checkbox `#activeOnlyCheck` (active-only toggle).
- **Data grid** — Columns: `Sl.No`, `Scope`, `Target`, `Shift`, `From`, `To`, `Active`. One row = one shift assignment at a given scope.
- **Connections** — Consumes shift masters from `/shifts`; resolved shift drives per-day expectations in `/attendance-log` (`Must Work Hour`), `/overtime-approval` and `/timesheet`.
- **Automation notes** — Mandatory fields marked `*`: Shift, Scope, Effective From — negative tests should submit with each missing. `Target` behaviour depends on chosen `Scope` (branch vs department vs employee lookup) — cover scope-switch re-rendering. `#activeOnlyCheck` is an id-based selector, rare and stable — use it. Precedence rule (most-specific wins, sub-department inheritance) is the key business assertion for E2E: create Company-level and Employee-level overlapping assignments and verify resolution.

Attendance Log (`/attendance-log`)

- **Purpose** — Day-wise processed attendance report per employee: entry/exit, worked vs must-work hours, overtime and balance hours, plus per-day session details. Page header reads `Attendance Report`, breadcrumb `HRMS > Attendance Log`.
- **UI elements** — Single `Filter` button on the `Attendance Log` card; no inline inputs were captured, so filter criteria (employee/date range etc.) live inside a modal/offcanvas opened by `Filter`.
- **Data grid** — Columns: `SL.No`, `IDNumber`, `Employee Name`, `Date`, `Period`, `Employee Status`, `Entry Time`, `Exit Time`, `Worked Hours`, `Over Time Hours`, `Balance Working Hours`, `Must Work Hour`, `Fixed Hours`, `Total Hours`, `Session Details`. One row = one employee-day of computed attendance.
- **Connections** — Aggregates raw punches from `/data-from-device` and `/add-visit-report`; corrected by approved `/regularization` requests; its worked-day rows feed the `/approval-operation` queue and OT minutes feed `/overtime-approval`; locked by `/attendance-finalization`.
- **Automation notes** — Grid `rowCount: 0` in the capture: data appears only after applying the `Filter` dialog — treat the filter as effectively mandatory and drive tests through it. `Session Details` is a drill-down cell (expect a per-row expander/dialog). Column set is wide — use horizontal-scroll-safe cell locators (header-index mapping) rather than nth-cell assumptions.

Data from device (`/data-from-device`)

- **Purpose** — Raw punch stream from biometric/attendance devices: every punch with recognition type, device, sync time, location and captured image, plus whether the punching person is registered in the system.
- **UI elements** — `Filter` button on the `Data From Device` card; no inline inputs (filter fields are inside the Filter modal/offcanvas).
- **Data grid** — Columns: `SL.No`, `ID Number`, `Employee Name`, `Punching Time`, `Punch Type`, `Recognition Type`, `Is Registered In System`, `Device Name`, `Punching Sync Time`, `Location`, `Image`. One row = one raw device punch event.
- **Connections** — Upstream source for `/attendance-log` (punches are paired into entry/exit and sessions). `Is Registered In System` flags orphan punches that will not post to any employee's log — a useful QA probe. `Location`/`Image` columns tie into the geofence/mobile capture story (`/geofences`).
- **Automation notes** — `rowCount: 0` until `Filter` is applied — same mandatory-filter pattern as `/attendance-log`. `Image` cell presumably renders a thumbnail/link — assert element presence, not text. Good page for data-driven verification after simulating device pushes: filter by a known employee + date and assert the punch row appears with correct `Device Name` and `Punch Type`.

Add Visit Report (`/add-visit-report`)

- **Purpose** — Log of mobile/field site-visit check-ins and check-outs with site, purpose and location evidence. Note the on-screen header is spelled `Add Vist Report` (UI typo) while the breadcrumb reads `Add visit report`; the inner card is titled `Attendance Log`.
- **UI elements** — `Filter` button; no inline inputs (filters in a modal/offcanvas).
- **Data grid** — Columns: `SL.No`, `IDNumber`, `Employee Name`, `CheckIn Time`, `CheckOut Time`, `Punch Type`, `Site Name`, `Purpose`, `Site Image`, `Mobile Location`, `Location`. One row = one site visit (check-in/check-out pair) by an employee.

- **Connections** — Second upstream punch source for `/attendance-log` (field attendance); validated against `/geofences` locations (mobile punching); `Site Image`/`Mobile Location` are the audit evidence columns.
- **Automation notes** — Match the header with the exact typo `Add Vist Report` if using text locators (or prefer the route/breadcrumb). `rowCount: 0` until `Filter` applied. `Site Image` renders media — assert presence/attribute. Card title `Attendance Log` duplicates the one on `/attendance-log` — never use that card title as a page discriminator; use URL or `h1` text.

Regularization (`/regularization`)

- **Purpose** — Employees/admins raise attendance correction requests (missed or wrong punches, manual entries) and track their approval status.
- **UI elements** — Two cards: `Raise A Correction` (form) and `Requests` (grid). Form fields: `Employee*` (select, default `-- select --`), `Date*` (date), `Type*` (select with options `Missed Punch`, `Wrong Punch`, `Missed Check In`, `Missed Check Out`, `Manual Entry`), `In Time` (datetime-local), `Out Time` (datetime-local), `Reason` (textarea), `Attachment` (file input); buttons `Submit` and `Clear`. Grid filters: text input `Search employee or reason...`, two selects (type/status style), and a from/to `date` pair.
- **Data grid** — Columns: `Sl.No`, `Employee`, `Date`, `Type`, `In`, `Out`, `Reason`, `Status`, `Action`. One row = one correction request with its lifecycle `Status` and per-row `Action` (approve/reject/withdraw style controls).
- **Connections** — An approved regularization adjusts the employee-day in `/attendance-log` (entry/exit/worked hours), which in turn changes what `/approval-operation`, `/overtime-approval` and `/attendance-finalization` see. `Pending` regularizations are exactly what the `Pending` column on `/attendance-finalization` gates on before locking a period.
- **Automation notes** — Mandatory: `Employee`, `Date`, `Type`. `In Time` / `Out Time` are `datetime-local` inputs — fill with `YYYY-MM-DDTHH:mm` strings. File `Attachment` — cover both with and without upload. `Clear` button resets the form (assert fields return to defaults). Status transitions in the `Requests` grid are the core E2E: submit → row appears with pending status → act via `Action` → status change reflected.

Overtime Approval (`/overtime-approval`)

- **Purpose** — Approval queue for computed overtime: per employee-day OT minutes with eligibility, payout and export-to-payroll status.
- **UI elements** — Card `Overtime Queue`. Filters: text input `Search employee or shift...`, two selects (status/eligibility style), and a from/to `date` pair. No page-level buttons — actions are per-row in the `Action` column.
- **Data grid** — Columns: `Sl.No`, `Employee`, `Date`, `Shift`, `OT Min`, `Eligible`, `Payout`, `Status`, `Exported`, `Action`. One row = one employee-day overtime record.
- **Connections** — OT minutes originate from `/attendance-log` (`Over Time Hours`) against the shift resolved by `/shift-roster`; `Exported` indicates handoff to payroll after approval; finalization (`/attendance-finalization`) is the downstream lock.
- **Automation notes** — No global submit — all mutations happen via row-level `Action` controls, so tests must locate rows by `Employee` + `Date` text. `Eligible`/`Exported` are boolean-style cells (badge/toggle) — assert rendered state. Filter by date range to bound the queue deterministically. Empty-capture row (`rowCount: 1`, blank cells) means an empty placeholder row exists — do not count rows to assert emptiness.

Attendance Finalization (`/attendance-finalization`)

- **Purpose** — Month-end pay-cycle control: start or refresh an attendance finalization run for a scope, with a cut-off date, then track run status and pending items before lock.
- **UI elements** — Two cards: `Start / Refresh A Pay-Cycle Run` (form) and `Runs` (grid). Form fields: `Month*` (select, `January` … `December`), `Year*` (select, `2023`–`2027`), `Scope*` (select: `Company` / `Branch` / `Department` / `Employee`), `Target *` (select, default `-- select branch --`), `Cut-Off*` (date); `Finalize` button. Grid filters: text input `Search branch, department or employee...` plus three selects (period/scope/status style).
- **Data grid** — Columns: `Sl.No`, `Period`, `Scope`, `Target`, `Cut-Off`, `Pending`, `Status`, `Action`. One row = one finalization run for a period+scope, with `Pending` (unresolved items blocking lock) and run `Status`.
- **Connections** — The terminal step of the sub-module: consumes the corrected `/attendance-log`, requires approval queues (`/regularization`, `/approval-operation`, `/approval-absent`, `/overtime-approval`) to be cleared (surfaced via `Pending`), and locks the period for payroll consumption.
- **Automation notes** — All five form fields are mandatory. `Target` select's default text changes with `Scope` (`-- select branch --` when Branch) — dynamic dependent dropdown, cover the cascade. `Finalize` on an already-run period should behave as "refresh" per the card title — test idempotency. `Pending > 0` vs `Status` interplay is the key business assertion (should a run finalize with pending regularizations?).

Geofence Locations (`/geofences`)

- **Purpose** — Maintain the geofence points (latitude/longitude/radius) that constrain mobile attendance punching, scoped to parts of the organisation.
- **UI elements** — `Add Location` button (also appears in the breadcrumb trail as a nav element); card `Locations`; filters: text input `Search location or target...` and two selects (scope/status style).
- **Data grid** — Columns: `Sl.No`, `Scope`, `Applies To`, `Name`, `Lat`, `Long`, `Radius`, `Status`, `Active`. One row = one geofence location and who it applies to.
- **Connections** — Governs validity of mobile punches surfacing in `/add-visit-report` (`Mobile Location`) and device/mobile punches in `/data-from-device` (`Location`). Same Scope/Applies-To pattern as `/shift-roster`.
- **Automation notes** — `Add Location` opens the create flow (likely modal with a map/lat-long form — verify at runtime). `Lat`/`Long`/`Radius` are numeric cells — assert numeric formats. `Status` vs `Active` are distinct columns (approval state vs on/off) — don't conflate in assertions. Placeholder `Search location or target...` is a stable locator.

Timesheet (`/timesheet`)

- **Purpose** — Daily reconciliation of attendance hours against task-management hours per employee ("Daily Worked Hours Vs Task Hours").
- **UI elements** — Card `Daily Worked Hours Vs Task Hours`; filters: text input `Search employee...`, two selects (shift/status style), and a from/to `date` pair. No action buttons — read-only comparison view.
- **Data grid** — Columns: `Sl.No`, `Date`, `Employee`, `Shift`, `Status`, `Attendance Hrs`, `Task Hrs`, `Tasks`. One row = one employee-day comparing attendance hours to logged task hours; `Tasks` is the drill-down into the task list behind `Task Hrs`.

- **Connections** — `Attendance Hrs` comes from `/attendance-log` (worked hours); `Task Hrs`/`Tasks` come from the Task Management module (cross-module link); `Shift` from `/shift-roster` resolution.
- **Automation notes** — Read-only page: tests are assertion-heavy, no mutations. Cross-module fixture needed (an attendance day plus logged task hours) to assert both columns on one row. `Tasks` cell likely expands or links — assert navigability. Empty placeholder row pattern applies (`rowCount: 1`, blank).

Attendance Report Pack (`/attendance-report-pack`)

- **Purpose** — Report-pack page under the Reports breadcrumb (`Reports > Attendance`) producing a paginated, exportable `Daily Register`.
- **UI elements** — `Export` button; a filter icon-button on the card header (opens the filter panel containing the captured 8 inputs: four selects + a from/to `date` pair + two more selects — branch/department/employee/status-style criteria); pagination controls `Previous` / `Next` with `Page 1` indicator; `Rows per page` selector with options `50 100 250 500` (screenshot shows default `100`); empty state text `No records.`
- **Data grid** — No table markup was captured in the empty state; once criteria are applied the `Daily Register` renders rows (one row = one employee-day register entry).
- **Connections** — Reporting face of `/attendance-log` data for a chosen period/scope; `Export` produces the downloadable register (payroll/audit handoff). Sits in the Reports menu group rather than Attendance.
- **Automation notes** — Assert the exact empty state `No records.` before filtering. The filter UI is behind an icon button (no text label) — locate by role/aria or position within the `Daily Register` card header next to `Export`. `Export` triggers a file download — use Playwright's `waitForEvent('download')`. Pagination test: change `Rows per page` and assert `Page 1` reset.

Approval Operation (`/approval-operation`)

- **Purpose** — Approval queue for worked-day attendance records: each employee-day with entry/exit, worked/OT/balance hours and late/early minutes awaiting an approval decision.
- **UI elements** — `Filter` button on the `Attendance Log` card (criteria live in the Filter modal/offcanvas); per-row `Approval` column carries the decision control.
- **Data grid** — Columns: `SL No`, `IDNumber`, `Employee Name`, `Branch`, `Date`, `Entry Time`, `Exit Time`, `Status`, `Period Name`, `Hours Employee Must Work`, `Worked Hours`, `Over Time Hours`, `Balance Hours`, `Entry Late Miutes`, `Exit Early Miutes`, `Approval`. One row = one worked employee-day pending approval. (Note the UI typos `Entry Late Miutes` / `Exit Early Miutes` — keep exact text in locators.)
- **Connections** — Rows come from processed `/attendance-log` days; approving a row posts it to `/approval-operation-report` (where `Fixed Hours` / `Final Hours` are recorded); clearing this queue reduces `Pending` on `/attendance-finalization`.
- **Automation notes** — `rowCount: 0` until `Filter` applied — mandatory-filter pattern. Column headers contain misspellings (`Miutes`) — copy header text verbatim into header-map helpers or tests will fail on "corrected" spelling. The `Approval` cell is the mutation point — locate row by `IDNumber` + `Date`. Card is titled `Attendance Log` like two other pages — discriminate by URL/`h1`.

Approval Operation Report (`/approval-operation-report`)

- **Purpose** — Post-approval register of worked-day records: what was approved, with fixed/final hours, permission-hour discounts and remarks; supports viewing details and deleting an approval.
- **UI elements** — `Filter` button on the `Approval Operation Report` card (criteria in Filter modal/offcanvas); per-row `Details` and `Delete` columns.
- **Data grid** — Columns: `SL No`, `IDNumber`, `Employee Name`, `Date`, `Entry Time`, `Exit Time`, `Balance Hours`, `Fixed Hours`, `Fixed Date`, `Discount from Permission Hours`, `Remarks`, `Final Hours`, `Details`, `Delete`. One row = one approved worked-day record.
- **Connections** — Downstream of `/approval-operation` (its approvals land here). `Discount from Permission Hours` links attendance to the leave/permission side of HRMS. `Delete` reverses an approval, returning the day to the `/approval-operation` queue.
- **Automation notes** — `rowCount: 0` until `Filter` applied. `Delete` is destructive — expect a confirm dialog; test cancel and confirm paths, then verify the record reappears in `/approval-operation`. `Details` opens a per-row drill-down (modal). Assert `Final Hours` equals expected computation after an approval round-trip from `/approval-operation`.

Approval Absent (`/approval-absent`)

- **Purpose** — Approval queue for absent days: employee-days with no attendance, listed with branch/department and the period's required hours, awaiting an absence-approval decision.
- **UI elements** — `Filter` button on the `Approval Absent` card (criteria in Filter modal/offcanvas); per-row `Approval` column carries the decision control.
- **Data grid** — Columns: `SL NO`, `IDNumber`, `Employee Name`, `Branch`, `Department`, `Date`, `Period Name`, `Period Type`, `Hours Employee Must Work`, `Approval`. One row = one absent employee-day pending approval.
- **Connections** — Absent days are the complement of worked days in `/attendance-log`; approving posts the record to `/approval-absent-report`; clearing the queue feeds the `Pending` gate on `/attendance-finalization`. Logically adjacent to the Leave module (an approved leave should not surface here).
- **Automation notes** — `rowCount: 0` until `Filter` applied. Header casing is inconsistent across sibling pages (`SL NO` here vs `SL No` / `SL.No` elsewhere) — normalise case-insensitively in shared helpers. Locate rows by `IDNumber` + `Date`; the `Approval` cell is the mutation point.

Approval Absent Report (`/approval-absent-report`)

- **Purpose** — Post-approval register of absent-day records with fixed/final hours and remarks; supports details drill-down and deletion of an approval.
- **UI elements** — `Filter` button on the `Approval Absent Report` card (criteria in Filter modal/offcanvas); per-row `Details` and `Delete` columns.
- **Data grid** — Columns: `SL No`, `IDNumber`, `Employee Name`, `Date`, `Period Name`, `Start Time`, `End Time`, `Balance Hours`, `Fixed Hours`, `Fixed Date`, `Remarks`, `Final Hours`, `Details`, `Delete`. One row = one approved absent-day record.
- **Connections** — Downstream of `/approval-absent`; `Delete` reverses the approval back into the `/approval-absent` queue; finalized figures roll into `/attendance-finalization` / payroll.
- **Automation notes** — `rowCount: 0` until `Filter` applied. Mirrors `/approval-operation-report` structurally — share a page-object base class; the differing columns are `Period Name`/`Start Time`/`End Time` here vs `Entry Time`/`Exit Time`/`Discount from Permission Hours` there. `Delete` destructive-action pattern: confirm dialog, then assert reappearance in `/approval-absent`.

Process flows

- **Shift setup → resolution**: `/shifts` (`New Shift` creates the master) → `/shift-roster` (`Assign` maps shift to Company/Branch/Department/Employee with `Effective From/To`; most-specific assignment wins, sub-departments inherit) → resolved shift drives `Must Work Hour` in `/attendance-log`, `Shift` in `/overtime-approval` and `/timesheet`.
- **Punch capture → daily log**: biometric punches land in `/data-from-device` (with `Is Registered In System` flagging orphans) and mobile field visits land in `/add-visit-report` (validated against `/geofences` locations) → punches are paired into entry/exit sessions and surface as employee-days on `/attendance-log`.
- **Correction loop**: missing/wrong punch noticed on `/attendance-log` → request raised on `/regularization` (`Raise A Correction`: Employee, Date, Type, In/Out, Reason, Attachment) → request approved via the `Requests` grid `Action` → corrected times reflected back in `/attendance-log`.
- **Worked-day approval chain**: filtered employee-days on `/approval-operation` → per-row `Approval` decision → approved record appears in `/approval-operation-report` with `Fixed Hours`/`Final Hours` (and `Discount from Permission Hours`) → `Delete` there reverses the approval back to the queue.
- **Absent-day approval chain**: absent employee-days on `/approval-absent` → per-row `Approval` decision → record appears in `/approval-absent-report` with `Fixed Hours`/`Final Hours` → `Delete` reverses back to the queue.
- **Overtime chain**: OT minutes computed on `/attendance-log` → queued on `/overtime-approval` (`OT Min`, `Eligible`, `Payout`) → approval sets `Status` → `Exported` marks handoff to payroll.
- **Month-end lock**: once regularizations and the three approval queues are cleared → `/attendance-finalization` `Start / Refresh A Pay-Cycle Run` (Month, Year, Scope, Target, Cut-Off) → `Finalize` → run tracked in `Runs` with `Pending` count and `Status` → period locked for payroll.
- **Reporting**: `/timesheet` reconciles `Attendance Hrs` vs Task Management `Task Hrs` per day; `/attendance-report-pack` (Reports menu) filters and `Export`s the paginated `Daily Register` for the finalized period.

Deep Dive — Leave Management

Overview

The Leave Management sub-module of the Progbiz HRMS ERP (https://hrms-erp.progbiz.in) covers the full leave lifecycle: HR first defines the building blocks (leave types, leave patterns, pattern-level policy rules, holiday calendars) and assigns them to organisational targets (branch / department / employee). Employees then raise leave requests, comp-off requests and encashment requests from self-service pages, while approvers process them on dedicated approval screens with bulk approve/reject and delegation support. Every approved movement posts to an append-only leave ledger, from which employee balances (Opening / Accrued / Used / Encashed / Available / Liability) are computed live. A Leave ↔ Attendance Sync engine reconciles approved leave with attendance rows, flags LOP days (IsLOP = 1) that flow to payroll at finalization, and can recalculate a period after retroactive holiday or policy changes. Continuity tooling (approval delegation and employee duty handover) keeps approvals moving when an approver is away. Finally, reporting surfaces — Leave Reports (Register / Balance / Utilization), Absence Analytics (Bradford Factor, absence rate by department) and a team Leave Calendar — give HR oversight of absence across the organisation.

Page Index

#	Page	Route	Purpose
1	Leave Types	`/leave-types`	Master list of leave types (half-day support, document requirement)
2	Leave Patterns	`/leave-patterns`	Define named leave patterns (bundles of leave rules)
3	Leave Policy Configuration	`/leave-policy`	Configure policy rules for a chosen leave pattern
4	Leave Assignment List	`/leave-assignment-list`	Assign leave patterns to targets (branch/department/employee)
5	Leave Request	`/leave-request-list`	Employee self-service: raise and track leave requests
6	Leave Approval	`/leave-approval`	Approver worklist: approve/reject leave requests, bulk actions, delegation
7	My Leave Policy	`/my-leave-policy`	Employee view of their assigned leave policy per year
8	My Leave Balance	`/leave-balances`	Employee balance detail per leave type, computed from the ledger
9	Leave Ledger	`/leave-ledger`	Append-only transaction ledger of all leave movements
10	Leave ↔ Attendance Sync	`/leave-attendance-sync`	Reconcile approved leave with attendance rows, LOP flags, recalculation
11	Leave Encashment	`/leave-encashment`	Employee self-service: request encashment of eligible balance
12	Encashment Approvals	`/leave-encashment-approval`	Approver worklist for encashment requests

#	Page	Route	Purpose
13	Leave Approval Delegation	`/leave-delegation`	View active & past approval delegations
14	Employee Duty Handover	`/employee-handover`	Hand an away employee's approvals/tasks to an assignee for a date window
15	Comp-Off	`/comp-offs`	Employee self-service: view/request comp-off credits
16	Comp-Off Management	`/comp-off-management`	HR/manager grant or reject comp-off credits for all employees
17	Holiday	`/holiday-list`	Holiday calendar master for the year
18	Holiday Assignment	`/holiday-assignment-list`	Assign holiday calendars to targets
19	Leave Reports	`/leave-reports`	Report hub: Register / Balance / Utilization reports
20	Absence Analytics	`/absence-analytics`	KPI dashboard: absence rate, Bradford Factor risk table
21	Leave Calendar	`/leave-calendar`	Visual calendar of leave across branch/department/employee

Pages

Leave Types (`/leave-types`)

- **Purpose** — Master screen for defining leave types (e.g. casual, medical) and their per-type behaviour flags: whether half-day leave is supported and whether a supporting document is required.
- **UI elements** — Two cards on one page: `Leave Types` (grid) and `Add LeaveType` (inline create form). Form fields: text input `Leave Type Name*` (id `leavetypename`), checkboxes `Support HalfDay` and `Need Document` (both share id `checkebox-sb` — a duplicate-id hazard for selectors). Buttons: `Save`, `Clear`.
- **Data grid** — Columns: `SlNo`, `Leave Type Name`, `Is Support Half Day`, `Is Need Document`, `Action`. A row = one leave type definition. Captured with 0 rows.
- **Connections** — Leave types defined here populate the type dropdowns across the module: leave requests (`/leave-request-list`), policy configuration (`/leave-policy`), encashment (`/leave-encashment`), reports (`/leave-reports` shows `Casual leave`, `Earned leave`, `Maternity leave`, `Medical leave`, `Paternity leave` in its Leave Type filter) and ledger/balance views.
- **Automation notes** — Create form is inline (no modal). The two checkboxes have the *same* id `checkebox-sb`; select by label text or DOM order, not by id. `Leave Type Name*` is mandatory (starred). Table renders headers even when empty — assert on row count, not table presence.

Leave Patterns (`/leave-patterns`)

- **Purpose** — List and create leave patterns — the named rule bundles that leave policy is configured against and that get assigned to employees.
- **UI elements** — Button `New Leave Pattern` (implies a create modal/form). Card: `Leave Patterns`.
- **Data grid** — Columns: `Sl.No`, `Leave Pattern Name`, `Details`, `Action`. A row = one leave pattern. Captured with 0 rows.

- **Connections** — Patterns created here are the input to `/leave-policy` (its only control is a `Leave Pattern` chooser) and to `/leave-assignment-list` (grid has a `Leave Pattern` column). An assigned pattern is what surfaces to the employee on `/my-leave-policy` ("Once HR assigns a leave pattern, your leave types, balances and rules appear here").
- **Automation notes** — No inputs captured on the list page itself, so `New Leave Pattern` almost certainly opens a modal — wait for the dialog after clicking. Empty grid renders headers only.

Leave Policy Configuration (`/leave-policy`)

- **Purpose** — Configure the policy rules (entitlements, accrual etc.) attached to a leave pattern.
- **UI elements** — Single card `Choose A Leave Pattern` containing one native `select` labelled `Leave Pattern` with placeholder option `Choose` (confirmed by screenshot). No other controls render until a pattern is picked.
- **Data grid** — None on initial load.
- **Connections** — Consumes patterns from `/leave-patterns`; the configured policy governs what employees see on `/my-leave-policy`, which types are encashable on `/leave-encashment` ("No leave type in your policy allows encashment" when none qualify), and accrual behaviour behind `Run Accrual` on `/leave-balances`.
- **Automation notes** — The page is gated on the pattern select: the policy form only appears after choosing a value, so tests must seed at least one leave pattern first. Initial state has no buttons or tables — a good precondition assertion. Breadcrumb: `HRMS » Leave Policy`.

Leave Assignment List (`/leave-assignment-list`)

- **Purpose** — Assign leave patterns to organisational targets so employees inherit a leave policy.
- **UI elements** — Buttons: `Filter`, `New Leave Assignment`. Card: `Leave Assignment`.
- **Data grid** — Columns: `Sl.No`, `Assignment Type`, `Assignment Target Name`, `Leave Pattern`, `Action`. A row = one pattern-to-target assignment (the `Assignment Type` column implies targets can be different entity kinds, mirroring `/holiday-assignment-list`).
- **Connections** — Consumes patterns from `/leave-patterns` (+ rules from `/leave-policy`). An assignment is the trigger that makes `/my-leave-policy` and `/leave-balances` non-empty for an employee, and enables raising typed requests on `/leave-request-list`.
- **Automation notes** — `New Leave Assignment` and `Filter` imply modal/panel interactions (no inline inputs captured). Empty grid renders headers only. Assert assignment visibility downstream via `/my-leave-policy` empty-state disappearing.

Leave Request (`/leave-request-list`)

- **Purpose** — Employee self-service list to raise new leave requests and track their approval status.
- **UI elements** — Button: `New Leave Request`. Card: `Leave Request List`.
- **Data grid** — Columns: `Sl.No`, `Leave Type`, `Start Date`, `End Date`, `Approval Status`, `Remarks`, `Action`. A row = one leave request by the logged-in employee with its current approval status. Captured with 0 rows.
- **Connections** — Requires the employee to have an assigned policy (types come from `/leave-types` via the assigned pattern). Submitted requests appear in the approver worklist `/leave-approval`; on approval they post to `/leave-ledger`, reduce `Available` on `/leave-balances`, and surface in `/leave-attendance-sync`, `/leave-reports`, `/absence-analytics` and `/leave-calendar`.

- **Automation notes** — `New Leave Request` opens the create flow (expect a modal/form — no inline inputs captured on the list). `Approval Status` is the key column for end-to-end assertions after acting on `/leave-approval`. Empty grid = headers only, no empty-state text.

Leave Approval (`/leave-approval`)

- **Purpose** — Approver worklist for pending leave requests, with bulk approve/reject and approval delegation.
- **UI elements** — Buttons: `Delegate approvals`, `Filter`, `Approve Selected`, `Reject Selected`, `Clear`. Helper text: "Bulk actions— tick the rows you want, then approve or reject them together". Inputs: a header row checkbox (select-all), three `select` filters all sharing id `selectbox`, and a text input id `custodianname` (delegate/custodian name — part of the `Delegate approvals` flow).
- **Data grid** — Columns: `Sl.No`, `Employee Name`, `Leave Type`, `Details`, `Leave Status`, `Action`. A row = one employee leave request awaiting the approver's decision, with a per-row checkbox for bulk actions.
- **Connections** — Fed by `/leave-request-list` submissions. Approval posts a ledger transaction to `/leave-ledger` (Source Ref) and updates `/leave-balances` and `/leave-attendance-sync`. `Delegate approvals` creates records visible in `/leave-delegation`; `/employee-handover` with `Cover approvals` routes this worklist to an assignee.
- **Automation notes** — Bulk flow: tick checkboxes → `Approve Selected` / `Reject Selected` (`Clear` resets selection). Three selects share id `selectbox` — disambiguate by index or wrapping label. `custodianname` is a stable id for the delegation input. Breadcrumb group here is `Leave » Leave Approval` (unlike most pages which sit under `HRMS`).

My Leave Policy (`/my-leave-policy`)

- **Purpose** — Read-only employee view of the leave policy (types, balances and rules) assigned to them for a chosen year.
- **UI elements** — `Year` number input + `Show` button. Empty state text: "No leave policy is assigned to you yet. Once HR assigns a leave pattern, your leave types, balances and rules appear here."
- **Data grid** — None captured (policy content renders only once a pattern is assigned).
- **Connections** — Populated by the `/leave-patterns` → `/leave-policy` → `/leave-assignment-list` chain. Companion self-service page to `/leave-balances`.
- **Automation notes** — The exact empty-state sentence is a reliable assertion for the "no policy assigned" precondition. `Year` is a plain `input[type=number]`; results load only after clicking `Show` (mandatory filter).

My Leave Balance (`/leave-balances`)

- **Purpose** — Employee's per-leave-type balance sheet for a year, computed live from the leave ledger, including monetary liability.
- **UI elements** — `Year` number input, buttons `Show` and `Run Accrual`. Card: `Balance Detail — 2026`. Footnotes: "Balances are computed live from the leave ledger." and "Liability = encashable available balance × (active monthly gross ÷ 30)."
- **Data grid** — Columns: `Sl.No`, `Leave Type`, `Opening`, `Accrued`, `Carried Fwd`, `Used`, `Encashed`, `Reserved`, `Available`, `Liability`. A row = one leave type's balance breakdown for the selected year. Captured empty with in-table message "No balances found." plus banner "No balances found for 2026."

- **Connections** — Derived entirely from `/leave-ledger` transactions; `Used` reflects approvals from `/leave-approval`, `Encashed` reflects `/leave-encashment` → `/leave-encashment-approval`, `Accrued` is driven by policy rules (`/leave-policy`) and the `Run Accrual` action. `Available` gates what `/leave-encashment` allows.
- **Automation notes** — Two empty-state strings to assert: page-level "No balances found for 2026." (year-interpolated) and table cell "No balances found.". `Run Accrual` is a state-mutating action — after it, expect new `Accrue` rows in `/leave-ledger` and non-zero `Accrued` here. Card title interpolates the year (`Balance Detail — <year>`).

Leave Ledger (`/leave-ledger`)

- **Purpose** — Append-only audit ledger of every leave transaction; the single source of truth balances are computed from.
- **UI elements** — Buttons: `Filter`, `Export`. Filters: employee text search (placeholder `All (search name / ID)`), one `select` (leave type/txn), and two `date` inputs (from/to). Banner: "Append-only — rows are never edited or deleted. Corrections post a new Adjust or Reverse entry referencing the original."
- **Data grid** — Columns: `Date`, `Employee`, `Leave Type`, `Txn`, `Days`, `Source Ref`, `Balance after`, `Posted by`. A row = one immutable ledger transaction (e.g. accrual, usage, encashment, adjust, reverse) with a running balance and originating reference. Captured empty ("No transactions.").
- **Connections** — Written to by approvals on `/leave-approval` and `/leave-encashment-approval`, by `Run Accrual` on `/leave-balances`, and by comp-off grants (`/comp-off-management`) where credits enter balance. Read by `/leave-balances` (live computation) and reporting pages.
- **Automation notes** — Never assert row edits/deletes — corrections appear as new `Adjust`/`Reverse` rows referencing the original (per the banner). `Source Ref` links a txn back to its originating request — useful for end-to-end traceability assertions. Employee filter placeholder `All (search name / ID)` is a stable selector hook. `Export` triggers a download.

Leave ↔ Attendance Sync (`/leave-attendance-sync`)

- **Purpose** — Reconciliation screen showing how each approved leave maps onto attendance rows, with LOP flags and a period recalculation tool.
- **UI elements** — Buttons: `Filter`, `Recalculate period`. Filters: one `number` input (year) and three `select`s (period/month, branch, department per the filter pattern used module-wide). Footnotes: "LOP days carry IsLOP = 1 on the attendance row and flow to payroll at finalization." and "Use Recalculate period after a retroactive holiday/policy change to re-run the engine for the filtered month."
- **Data grid** — Columns: `Leave Ref`, `Employee`, `Dates`, `Attendance rows`, `LOP`, `Sync status`. A row = one approved leave and its attendance-side materialisation. Captured empty ("No leave in this period.").
- **Connections** — Consumes approved leave from `/leave-approval` (via the ledger). `LOP` output feeds payroll at finalization (cross-module). Retroactive changes to `/holiday-list` or `/leave-policy` are the documented trigger for `Recalculate period`.
- **Automation notes** — `Recalculate period` is a bulk recomputation for the *filtered month* — set filters first. Good regression test: approve leave overlapping a newly added holiday, run `Recalculate period`, assert `Sync status`/`LOP` changes. Empty state text: "No leave in this period."

Leave Encashment (`/leave-encashment`)

- **Purpose** — Employee self-service page to convert encashable leave balance into money and track those requests.
- **UI elements** — Card `Request Encashment` with: `Leave Type` select (placeholder `Choose`), `Days` number input, `Per Day Rate` number input, computed display `Amount: 0`, and button `Submit Request`. Card `My Requests` for history. Inline notice when nothing is eligible: "No leave type in your policy allows encashment."
- **Data grid** — `My Requests` columns: `SlNo`, `Leave Type`, `Days`, `Amount`, `Status`. A row = one encashment request by the logged-in employee. Captured empty ("No requests.").
- **Connections** — Eligibility comes from the assigned policy (`/leave-policy` via `/leave-assignment-list`) and available balance on `/leave-balances`. Submitted requests go to `/leave-encashment-approval`; approval posts to `/leave-ledger` and increments `Encashed` on `/leave-balances` (which also computes `Liability` from encashable balance).
- **Automation notes** — `Amount` is auto-computed (Days × Per Day Rate) — assert the recalculation on input change. The "No leave type in your policy allows encashment" notice is the precondition empty-state; seeding an encashable type in the policy is required before the form is usable. Status transitions in `My Requests` after acting on the approval page are the key E2E assertion.

Encashment Approvals (`/leave-encashment-approval`)

- **Purpose** — Approver worklist to approve or reject employee encashment requests.
- **UI elements** — Button: `Filter`. Five `select` filter controls (consistent with year/branch/department/type/status filtering). Card: `Encashment Requests`.
- **Data grid** — Columns: `SlNo`, `Employee`, `Leave Type`, `Days`, `Amount`, `Status`, `Action`. A row = one pending/processed encashment request across employees. Captured empty ("No requests.").
- **Connections** — Fed by `/leave-encashment` submissions. Approval posts an encashment transaction to `/leave-ledger` and updates `Encashed`/`Available`/`Liability` on `/leave-balances`.
- **Automation notes** — Row-level approve/reject actions live in the `Action` column (no bulk buttons captured, unlike `/leave-approval`). All five filter selects are unlabelled native selects — target by position or surrounding text. Empty state: "No requests."

Leave Approval Delegation (`/leave-delegation`)

- **Purpose** — Registry of leave-approval delegations: who has handed approval authority to whom and for which date window.
- **UI elements** — Card: `Active & Past Delegations`. No create buttons on this page — delegations are created via `Delegate approvals` on `/leave-approval` (which carries the `custodianname` input).
- **Data grid** — Columns: `SlNo`, `From`, `To`, `From Date`, `To Date`, `Active`, `Action`. A row = one delegation record with its validity window and active flag. Captured empty ("No delegations.").
- **Connections** — Created from `/leave-approval` (`Delegate approvals`); while active, the delegate sees/actions the `/leave-approval` worklist. Related to (but distinct from) `/employee-handover`, which covers broader duty handover.
- **Automation notes** — Read/manage-only page: to test creation, drive the `Delegate approvals` flow on `/leave-approval` and assert the new row here. `Active` column + `Action` column suggest deactivation is done per-row. Empty state: "No delegations."

Employee Duty Handover (`/employee-handover`)

- **Purpose** — HR tool to hand an away employee's HRMS approvals and HRMS/recruitment tasks to an assignee for a date window (explicitly excludes Sales/CRM duties).
- **UI elements** — Card `Set Up Handover`: selects `Employee going away` (`-- Select employee --`) and `Assign duties to` (`-- Select assignee --`), `From` / `To` date inputs, checkboxes `Cover approvals` (id `ca`), `Cover HRMS tasks` (id `ct`), `Active` (id `active`), textarea `Note (optional)`, buttons `Save` / `Clear`. Helper text: "Hand an away employee's HRMS approvals and HRMS/recruitment tasks to an assignee for a date window. Sales/CRM duties are never handed over." Card `All Handovers` (grid).
- **Data grid** — Columns: `#`, `From`, `To`, `From Date`, `To Date`, `Covers`, `Active`, `Action`. A row = one handover record showing what is covered and whether it is active. Captured empty ("No handovers.").
- **Connections** — With `Cover approvals` ticked, the assignee effectively receives the away employee's `/leave-approval` (and other HRMS approval) queues — the broader sibling of `/leave-delegation`. `Cover HRMS tasks` extends to HRMS/recruitment task queues outside this sub-module.
- **Automation notes** — Inline create form with stable checkbox ids `ca`, `ct`, `active` — good selectors. Date window (`From`/`To`) plus `Active` govern effectiveness; assert the `Covers` cell reflects the ticked checkboxes. Empty state: "No handovers."

Comp-Off (`/comp-offs`)

- **Purpose** — Employee self-service view of compensatory-off credits earned by working off-days/holidays, with the ability to request a credit manually.
- **UI elements** — Button: `Request Comp-Off`. Card: `My Comp-Off Credits`. Helper text: "Credits are earned automatically when you work an approved off-day / holiday, or you can request one below."
- **Data grid** — Columns: `Earned`, `Source`, `Days`, `Expiry`, `Status`, `Action`. A row = one comp-off credit (auto-earned or requested) with its expiry and grant status. Captured empty with in-table message "No comp-off credits yet. Worked an off-day? Request one above."
- **Connections** — Requests raised here appear in `/comp-off-management` with `Status Requested` for grant/reject. Auto-earn is driven by attendance on days marked as holidays (`/holiday-list`) or off-days. Granted credits become usable leave balance (comp-off usage then flows through the request → approval → ledger chain).
- **Automation notes** — `Request Comp-Off` opens the request flow (no inline inputs captured — expect a modal). Distinct, quirky empty-state sentence is a reliable assertion. Status column transitions (`Requested` → granted/rejected) after acting on `/comp-off-management` are the E2E check.

Comp-Off Management (`/comp-off-management`)

- **Purpose** — HR/manager console listing comp-off credits for all employees, to grant or reject pending requests.
- **UI elements** — Buttons: `Grant`, `Reject` (row-level actions). Checkbox filter `Pending grant only` (id `reqOnly`). Card: `Comp-Off Credits — All Employees`.
- **Data grid** — Columns: `SlNo`, `Employee`, `Earned`, `Source`, `Days`, `Expiry`, `Status`, `Action`. A row = one employee's comp-off credit. Captured with 1 live row: `Akshay ASK112` | earned `19/07/26` | source `worked in holiday` | `1` day | expiry `17/10/26` | status `Requested` | actions `Grant Reject`.
- **Connections** — Fed by `/comp-offs` requests and automatic earning (source `worked in holiday` ties back to `/holiday-list` + attendance). Granting turns the credit into usable balance (visible downstream in balances/ledger); the employee sees the status change on `/comp-offs`.
- **Automation notes** — `reqOnly` checkbox is a stable id for the pending-only filter. `Grant`/`Reject` are per-row buttons inside the `Action` cell of `Requested` rows. Real data exists on this environment

(Akshay ASK112) — tests should not assume an empty grid. `Source` values like `worked in holiday` are free-text-ish labels; match loosely.

Holiday (`/holiday-list`)

- **Purpose** — Maintain the organisation's holiday calendar(s) for the year — the base data for working-day maths, comp-off earning and attendance sync.
- **UI elements** — Buttons: `Calendar` (view toggle), `Export`, `New Holiday`. Search input with placeholder `Search by Holiday Name`. Card: `Holiday Calendar – 2026`.
- **Data grid** — Columns: `Sl.No`, `Holiday Name`, `Dates`, `Calendar`, `Action`. A row = one holiday with the calendar it belongs to. Captured with 1 row: `Onam` | `24/08/2026` | calendar `State - Kerala`.
- **Connections** — Holiday calendars are assigned to targets via `/holiday-assignment-list`. Holidays feed comp-off earning (`worked in holiday` source on `/comp-off-management`), the `/leave-attendance-sync` engine (retroactive holiday changes require `Recalculate period`), and scheduled-day denominators in `/absence-analytics`.
- **Automation notes** — `New Holiday` implies a create modal. `Calendar` button toggles a calendar visualisation of the same data. The `Calendar` *column* (e.g. `State - Kerala`) shows holidays are grouped into named calendars — create tests must pick one. Search input placeholder is a stable selector. `Export` triggers a download.

Holiday Assignment (`/holiday-assignment-list`)

- **Purpose** — Assign holiday calendars to organisational targets so the right holiday set applies to each employee group.
- **UI elements** — Buttons: `Filter`, `New Holiday Assignment`. Card: `Holiday Assignment List`.
- **Data grid** — Columns: `Sl.No`, `Assignment Type`, `Assigned Target Name`, `Action`. A row = one calendar-to-target assignment. Captured with 0 rows.
- **Connections** — Consumes calendars from `/holiday-list`. The effective holiday set per employee drives `/leave-attendance-sync` results, comp-off auto-earning and working-day calculations in reports/analytics. Structural twin of `/leave-assignment-list`.
- **Automation notes** — `New Holiday Assignment` implies a modal (no inline inputs captured). Empty grid renders headers only. Note the header wording difference vs. leave assignments: `Assigned Target Name` here vs. `Assignment Target Name` there — keep selectors page-specific.

Leave Reports (`/leave-reports`)

- **Purpose** — Report hub for leave data: pick a report type (Register / Balance / Utilization), set filters, and run it.
- **UI elements** — Report-type buttons: `Register`, `Balance`, `Utilization`; action button `Filter`. Filters: `Year` number input, `Branch` select (`All Branches`, `Main Branch`, `Chennai`), `Department` select (`All Departments`, `2D`, `DigitalMarkrting`, `Dotnet`, `Sales`, `SEO`), `Leave Type` select (`All Types`, `Casual leave`, `Earned leave`, `Maternity leave`, `Medical leave`, `Paternity leave`). Card: `Leave Register`. Hint: "Pick a report, set your filters, then run." / "Choose a report type and click Run Report."
- **Data grid** — None on initial load; the result grid renders only after running a report.
- **Connections** — Reads the data produced by the request/approval/ledger chain (`/leave-request-list` → `/leave-approval` → `/leave-ledger`) and balances. Leave Type options mirror `/leave-types` masters; Branch/Department options come from org masters shared with `/absence-analytics` and `/leave-calendar`.

- **Automation notes** — Running a report is a two-step interaction: click a report-type button, then `Filter`/run — the initial card shows the "Choose a report type and click Run Report." placeholder to assert against. Note the misspelled department option `DigitalMarkrting` — copy it exactly in selectors. Card title changes with the selected report (default `Leave Register`).

Absence Analytics (`/absence-analytics`)

- **Purpose** — Analytics dashboard for absence health: KPI tiles, monthly trend, per-department absence rate and a Bradford Factor risk table.
- **UI elements** — Buttons: `Filter`, `Export`. Filters: `Year` number input, `Branch` select, `Department` select. KPI tiles: `Absence rate` (healthy, "% of scheduled days"), `Unplanned share` (watch, "sick + same-day alerts"), `Bradford alerts` ("score > 250 YTD"), `Avg days / employee`. Chart cards: `Monthly Absence Trend` (Jan–Dec, % of yearly total, current month highlighted) and `Absence Rate By Department` ("Absence rate = leave days ÷ scheduled working days (per department)"). Risk card: `Bradford Factor — Highest Risk (S² × D)` with thresholds `Alert > 250`, `Monitor > 125`, `Normal`.
- **Data grid** — Bradford table columns: `SL No`, `Employee`, `Department`, `Spells (S)`, `Days (D)`, `Score (S²·D)`, `Signal`. A row = one employee's Bradford Factor score and risk signal. Captured with 1 row: `Akshay` | `Dotnet` | S=1, D=1, Score=1 | `Normal`.
- **Connections** — Aggregates approved leave (ledger/attendance-sync data) against scheduled working days (holiday calendars from `/holiday-list`). Shares Branch/Department filter masters with `/leave-reports` and `/leave-calendar`.
- **Automation notes** — KPI values render as plain numbers/percentages in headers (`0%`, `0`, `0.01`) — snapshot-style assertions are fragile; prefer asserting tile labels + numeric format. Bradford `Signal` values (`Normal`, and by thresholds `Monitor`/`Alert`) are deterministic from Spells/Days — computable expected values. `Export` downloads the risk table. Live data exists (Akshay/Dotnet).

Leave Calendar (`/leave-calendar`)

- **Purpose** — Visual calendar showing who is on leave, filterable by branch, department and employee name.
- **UI elements** — Filters: `Branch` select (`All Branches`), `Department` select (`All Departments`), `Employee` text input (placeholder `Type a name to filter…`), button `Show`. Screenshot confirms the calendar body loads asynchronously with a spinner and "Loading calendar…" text; breadcrumb `HRMS » Leave Management » Leave Calendar`.
- **Data grid** — None; the content is a calendar visualisation, not a table.
- **Connections** — Renders approved leave from the request/approval chain plus holidays; useful to managers before approving on `/leave-approval` (overlap awareness). Shares Branch/Department masters with `/leave-reports` and `/absence-analytics`.
- **Automation notes** — Async load: wait for the "Loading calendar…" spinner to disappear before asserting calendar content — the capture caught it mid-load, so allow generous timeouts. `Show` re-queries with current filters. Employee filter placeholder `Type a name to filter…` is a stable selector.

Process flows

- **Policy setup (HR admin)**: define types in `/leave-types` → create a pattern in `/leave-patterns` → configure its rules in `/leave-policy` (page is gated on choosing a pattern) → assign the pattern to branch/department/employee in `/leave-assignment-list` → employee's `/my-leave-policy` empty state ("No leave policy is assigned to you yet…") is replaced by their types, balances and rules.

- **Holiday setup**: create holidays/calendars in `/holiday-list` (e.g. `Onam`, `State - Kerala`) → assign calendars to targets in `/holiday-assignment-list` → holidays drive working-day maths in attendance sync, comp-off earning and analytics; a retroactive holiday change requires `Recalculate period` on `/leave-attendance-sync`.
- **Leave request lifecycle**: employee raises a request via `New Leave Request` on `/leave-request-list` → it appears in the approver worklist `/leave-approval` (single or bulk `Approve Selected` / `Reject Selected`) → approval posts an immutable transaction to `/leave-ledger` (with `Source Ref` back to the request) → `/leave-balances` recomputes `Used`/`Available` live from the ledger → `/leave-attendance-sync` materialises the leave onto attendance rows (LOP days get `IsLOP = 1` and flow to payroll) → the leave shows up in `/leave-reports`, `/absence-analytics` and `/leave-calendar`.
- **Accrual**: `Run Accrual` on `/leave-balances` applies the pattern's accrual rules → posts `Accrue` transactions to `/leave-ledger` → `Accrued` and `Available` columns update.
- **Encashment lifecycle**: employee submits on `/leave-encashment` (`Leave Type` must be encashable per policy; `Amount` = Days × Per Day Rate) → request lands in `/leave-encashment-approval` → approval posts to `/leave-ledger` → `/leave-balances` shows it under `Encashed` and `Liability` shrinks (Liability = encashable available balance × monthly gross ÷ 30).
- **Comp-off lifecycle**: employee works an approved off-day/holiday (auto-earn) or clicks `Request Comp-Off` on `/comp-offs` → credit appears as `Requested` in `/comp-off-management` (e.g. Akshay ASK112, source `worked in holiday`, with expiry) → HR clicks `Grant` or `Reject` → granted credit becomes usable balance until its `Expiry` date.
- **Approver continuity**: approver clicks `Delegate approvals` on `/leave-approval` (custodian name input) → delegation recorded in `/leave-delegation` (`From`/`To`/date window/`Active`) → alternatively HR sets up a broader `/employee-handover` with `Cover approvals` / `Cover HRMS tasks` for a date window (Sales/CRM duties never handed over) → the assignee works the away employee's approval queues.
- **Oversight loop**: `/leave-reports` (Register / Balance / Utilization) and `/absence-analytics` (absence rate, unplanned share, Bradford Factor $S^2 \cdot D$ with Alert > 250 / Monitor > 125) surface aggregate outcomes of all the flows above; `/leave-calendar` gives managers a visual overlap check before approving new requests.

Deep Dive — My Workspace (ESS)

> *Sub-module study for Playwright automation · App: https://hrms-erp.progbiz.in · Tenant `Hrms`*
>
> *Source: live crawl 2026-07-20 (11 pages). Raw captures in `hrms/data/pages/`, shots in `hrms/screenshots/ess/`.*

Overview

My Workspace is the employee-facing portal (Employee Self-Service / ESS). It is the counterpart to the admin HRMS modules: instead of HR acting *on* employees, the logged-in employee acts on their *own* record. From here an employee sees a personal dashboard, views/edits their profile (edits go to HR as approval requests), applies for and tracks leave, views and regularizes their own attendance, raises overtime, manages geo work-locations, uploads personal documents, requests and acknowledges letters/certificates, views payslips, sets up duty handovers, and checks their probation status.

Every self-service action that changes official data is **routed to an approval queue** rather than applied directly — profile edits → HR approval (`/ess/requests` → `/approvals`), leave → `/leave-approval`, regularization/OT → attendance approval queues, work-locations → geofence approval. So ESS is primarily a **request-origination surface**; the authoritative processing happens in the admin sub-modules. All ESS routes live under `/ess/*` except `/my-handover`.

Page Index

#	Page	Route	Purpose
1	My Workspace (Dashboard)	`/ess`	Personal home: KPI tiles, profile card, quick actions, holidays, birthdays
2	My Profile	`/ess/profile`	View own master data; submit field-change requests to HR
3	My Requests	`/ess/requests`	Track status of submitted profile-change requests
4	My Leave	`/ess/leave`	View leave balances and apply for / track leave
5	My Handover	`/my-handover`	Set up duty handover to a colleague during absence
6	My Attendance	`/ess/attendance`	View own daily attendance; regularize / raise OT
7	My Locations	`/ess/locations`	Register geo work-locations (map) for approval
8	My Documents	`/ess/documents`	Upload personal documents with expiry tracking
9	My Letters	`/ess/letters`	Request letters/certificates; acknowledge published ones
10	My Pay	`/ess/payslips`	View monthly payslips
11	My Probation	`/ess/probation`	View own probation status/reviews

Pages

My Workspace — Dashboard (`/ess`)

- **Purpose** — the employee's landing page inside My Workspace; a read-only snapshot with shortcuts into the rest of ESS. Content loads asynchronously after a "Loading your workspace…" placeholder (relevant for test waits).

- **UI elements** —
  - **4 KPI tiles:** `Leave Available` (e.g. "0 days"), `Today's Attendance` (e.g. "Not marked"), `Pending Requests` (count), `Letters to Acknowledge` (count).
  - **My Profile** card: avatar, name, designation, `Employee Code`, `Branch`, `Email`, `Phone`, `Reports To`, `Joined`.
  - **Quick Actions** buttons: `Apply Leave`, `My Attendance`, `Payslips`, `Documents`, `Letters`, `Profile`.
  - **Upcoming Holidays** widget (holiday name + date, e.g. "Onam — 24-Aug").
  - **Birthdays This Week** widget.
- **Connections** — Quick Actions deep-link to `/ess/leave`, `/ess/attendance`, `/ess/payslips`, `/ess/documents`, `/ess/letters`, `/ess/profile`. KPI tiles reflect data owned by `/leave-balances`, `/attendance-log`, `/ess/requests`, `/ess/letters`. Upcoming Holidays is fed by `/holiday-list` + `/holiday-assignment-list`.
- **Automation notes** — wait for the loading placeholder to clear before asserting (`waitForFunction` on body text not matching `/loading/i`). KPI tiles and Quick Actions are the stable assertion targets; the widgets are personalised to the logged-in user (`Vismaya`, code `PB1053`, Main Branch).

My Profile (`/ess/profile`)

- **Purpose** — read-only view of the employee's own master record, plus a controlled form to request changes to editable fields. Edits are **not applied directly** — they become HR approval requests.
- **UI elements** —
  - **Overview** section (read-only): `Employee Code` (PB1053), `Branch`, `Department`, `Designation`, `Reports To`, `Joined`, `Confirmed`, `Phone`.
  - **Request A Change** form with editable fields: `First Name`, `Last Name`, `Email`, `National / ID Number`, `Passport No`, `Blood Group`, and a `Reason` field.
  - Buttons: `Submit Change Request`, `View My Requests`.
  - Helper text: "*Edits to these fields are submitted to HR for approval before they take effect.*"
- **Data grid** — none (form + read-only overview).
- **Connections** — `Submit Change Request` creates a record shown on `/ess/requests`; `View My Requests` navigates there. The approval is actioned by HR via the workflow engine (`/approvals`). The authoritative record lives in admin `/employees`.
- **Automation notes** — this is the ESS write-path with a clear before/after: submit a change → assert it appears on `/ess/requests` with a pending status. Field labels above are reliable locators.

My Requests (`/ess/requests`)

- **Purpose** — lists the employee's submitted **Profile Change Requests** and their approval status.
- **UI elements** — `Profile Change Requests` card. Empty state: "*You have no change requests.*"
- **Data grid** — request list (populated once a change is submitted from `/ess/profile`).
- **Connections** — receives records from `/ess/profile`; each request is decided in the approval workflow (`/approvals`). Once approved, the change reflects on `/employees` / `/ess/profile`.
- **Automation notes** — assert the empty-state string on a fresh account; after submitting from `/ess/profile`, assert the new row appears here. Pairs with `/ess/profile` for the round-trip test.

My Leave (`/ess/leave`)

- **Purpose** — the employee's leave hub: see balances and apply for leave.

- **UI elements** —
  - `Show` (refresh by year) and `Submit Request` buttons.
  - **Apply For Leave** form: leave-type select, `From`/`To` date pickers, `halfday` toggle, `Reason` textarea, year (number) input.
  - **Balances 2026** card and **My Leave Requests 2026** card.
- **Data grid** — two tables: **Balances** [`Leave Type`, `Balance`, `Reserved`, `Available`] and **Requests** [`Type`, `From`, `To`, `Days`, `Status`].
- **Connections** — `Submit Request` creates a leave request that also surfaces on admin `/leave-request-list` and is approved on `/leave-approval`; approval posts to `/leave-ledger` and updates `/leave-balances`. Balances shown here mirror `/leave-balances`. Half-day support depends on the `Is Support Half Day` flag from `/leave-types`.
- **Automation notes** — the primary ESS→admin E2E: apply here → approve on `/leave-approval` → verify balance decremented + ledger entry. Balance table's `Reserved` vs `Available` columns reflect pending requests.

My Handover (`/my-handover`)

- **Purpose** — the employee sets up a **duty handover** to a colleague who will cover their responsibilities during an absence.
- **UI elements** — **Set Up Handover** form: `Assignee` select, `From`/`To` date pickers, `Covers` (ca/ct) selects, `Reason` textarea; `Save` / `Clear` buttons. **My Handovers** card lists existing handovers.
- **Data grid** — [`#`, `Assignee`, `From`, `To`, `Covers`, `Active`, `Action`].
- **Connections** — the ESS-side twin of admin `/employee-handover`; a handover can cover an approver so that leave/approvals route to the delegate (relates to `/leave-delegation`, `/leave-approval`).
- **Automation notes** — self-service version has an `Assignee` picker (vs admin's `From`/`To` employees). Save then assert the new row in **My Handovers** with `Active` = yes.

My Attendance (`/ess/attendance`)

- **Purpose** — the employee views their own daily attendance history and raises corrections.
- **UI elements** — `Show` (date-range filter with `From`/`To` dates), `Regularize`, `Raise OT` buttons. **Attendance History** card.
- **Data grid** — [`Date`, `Entry`, `Exit`, `Worked (min)`, `OT (min)`, `Status`].
- **Connections** — `Regularize` feeds admin `/regularization`; `Raise OT` feeds `/overtime-approval`. The underlying data is the same computed log shown in admin `/attendance-log`. Punches originate from `/data-from-device`, mobile/geo punches (`/geofences`, `/ess/locations`) and `/add-visit-report`.
- **Automation notes** — must click `Show` (apply date range) before rows render. `Regularize`/`Raise OT` open request flows whose results appear in the corresponding admin approval queues — good for ESS→admin assertions.

My Locations (`/ess/locations`)

- **Purpose** — the employee registers **geo work-locations** (e.g. home, a site) that, once approved, become valid mobile punch zones.
- **UI elements** — **Add A Work Location** form: `Name` (e.g. "Home, Site A"), `Lat`/`Long`/`Radius` numbers, an address search box that moves a **Leaflet map**; `Use my location` and `Submit for approval` buttons. **My Locations** card.

- **Data grid** — [`Sl.No`, `Name`, `Lat`, `Long`, `Radius`, `Status`].
- **Connections** — `Submit for approval` sends the location to admin `/geofences`; once active it gates geo-based attendance punches that feed `/attendance-log` / `/ess/attendance`.
- **Automation notes** — contains a third-party **Leaflet** map (external `leafletjs.com` asset) — avoid asserting on map internals; assert on the form fields and the resulting row `Status` (e.g. Pending → approved geofence). `Use my location` needs geolocation permission (may not be grantable headless).

My Documents (`/ess/documents`)

- **Purpose** — the employee uploads personal documents (IDs, certificates) with category and expiry tracking.
- **UI elements** — **Upload A Document** form: `Type` select, `Number` text, `Category`, `Expiry` date, `file` input; `Upload` button. **Documents** card.
- **Data grid** — [`Type`, `Number`, `Category`, `Expiry`, `Status`].
- **Connections** — uploaded docs attach to the employee master (`/employees`) and are visible to HR; expiry drives compliance reminders.
- **Automation notes** — a `file` input is present — use `setInputFiles` with a fixture (reuse `erp/common/sample-document.txt`). Assert the uploaded row appears with the chosen `Type`/`Expiry`.

My Letters (`/ess/letters`)

- **Purpose** — the employee requests HR letters/certificates and acknowledges letters HR has published to them.
- **UI elements** — **Request A Letter / Certificate** card and **Published Letters** card.
- **Data grid** — [`Letter`, `Type`, `Issued`, `Acknowledged`].
- **Connections** — HR generates letters via `/letters/templates` → `/letters/generate`; published letters land here for the employee to acknowledge (the `Letters to Acknowledge` KPI on `/ess` counts these).
- **Automation notes** — the acknowledge action flips the `Acknowledged` column and decrements the dashboard KPI — a clean before/after assertion. Requesting a letter creates an HR-side task.

My Pay (`/ess/payslips`)

- **Purpose** — the employee views their monthly payslips.
- **UI elements** — `Show` button (year/period filter, number input). **Payslips** card.
- **Data grid** — [`Period`, `Basic`, `Deductions`, `Net`, `Payable`, `Paid`].
- **Connections** — payslip rows are produced by admin `/employee-salary-process` (which consumes `/salary-revisions`, `/employee-deduction`, and finalized attendance from `/attendance-finalization`). Leave encashment (`/leave-encashment`) and LOP (`/leave-attendance-sync`) also affect the figures.
- **Automation notes** — click `Show` to load a period before asserting. Columns tie directly back to the salary-process output — useful as the terminal assertion of a payroll E2E.

My Probation (`/ess/probation`)

- **Purpose** — the employee's read-only view of their own probation status and reviews.

- **UI elements** — a status panel. Empty state for a confirmed employee: "*You are not currently on probation.*"
- **Data grid** — none when not on probation; shows review checkpoints when active.
- **Connections** — mirrors admin `/hrms/probation` (dashboard) and `/hrms/probation-report`; probation is started for new hires via `/hrms/probation` using `/hrms/probation-templates`.
- **Automation notes** — assert the "not currently on probation" string for the confirmed test user (`Vismaya` is `Confirmed`). To test the populated state, an employee with an active probation record is required.

Process flows

- **Profile change round-trip:** `/ess/profile` (edit + Reason → `Submit Change Request`) → record on `/ess/requests` (Pending) → HR decides via `/approvals` → approved change reflects on `/ess/profile` & `/employees`.
- **Leave self-service:** `/ess/leave` (Apply For Leave) → admin `/leave-request-list` / `/leave-approval` → on approve: `/leave-ledger` entry + `/leave-balances` decrement → balances update back on `/ess/leave` and the `/ess` KPI tile.
- **Attendance correction:** `/ess/attendance` (`Regularize` / `Raise OT`) → admin `/regularization` / `/overtime-approval` → approved correction updates the day log shown on `/attendance-log` & `/ess/attendance`.
- **Work-location approval:** `/ess/locations` (map + `Submit for approval`) → `/geofences` (approve) → active geofence enables mobile punches → punches surface in `/ess/attendance`.
- **Letters:** HR `/letters/generate` → published to `/ess/letters` → employee acknowledges → `/ess` "Letters to Acknowledge" KPI decrements.
- **Pay visibility:** `/employee-salary-process` (admin) → payslip rows on `/ess/payslips`.
- **Handover during absence:** `/my-handover` (set assignee) ↔ admin `/employee-handover` / `/leave-delegation` so approvals/duties are covered while the employee is away.

Automation quick-reference (ESS)

Behaviour	Where	Test implication
Async "Loading…" placeholder	`/ess`, `/ess/probation`	wait for placeholder to clear before asserting
Filter/`Show` before data renders	`/ess/attendance`, `/ess/payslips`, `/ess/leave`	click `Show` / apply dates first
`file` upload input	`/ess/documents`	`setInputFiles` with a fixture
Leaflet map + geolocation	`/ess/locations`	assert form/row, not map; geolocation may be blocked headless
Empty states as assertions	`/ess/requests` ("no change requests"), `/ess/probation` ("not currently on probation")	reliable on the fresh/confirmed test user
Write actions create approval requests	profile, leave, attendance, locations	verify the request appears in the matching admin queue, not an immediate data change